PAWN: Preference-Aware Semantic NFT for Online Reinforcement Learning of Flow-based VLAs

Haomin Zuo[1 †] **Wenzhan Li**[1,2 †] **Ruimao Zhang**[1] **Yulan Guo**[1,2 *]
[1]Sun Yat-sen University [2]Shenzhen Loop Area Institute
Code: https://github.com/ZzzzzZhhmm/pawn
[†]Equal Contribution. [*]Corresponding author.

Abstract: Online reinforcement learning for flow-based manipulation policies is challenging because multi-step solver-based action generation makes the action likelihoods required by standard policy-ratio objectives difficult to access. NFT-style methods offer a promising likelihood-free route for local policy optimization, and recent embodied extensions have begun to adapt this idea to flow-based robot policies. However, several structural issues remain: trajectory-level outcomes provide noisy and misaligned credit for action chunks and their intermediate solver states; full-space mirrored comparisons ignore the compositional geometry of robot actions; and local updates may still follow noisy behavior-sampled continuations without explicitly identifying better alternatives, which may be guided toward the manifold of suboptimal actions. We present **P**reference-**AW**are Semantic **N**FT (**PAWN**), a unified NFT-style online RL framework that addresses these issues through structured local preference optimization. PAWN derives implicit chunk-level preference signals by lifting solver-step density comparisons to chunk-path scores, reorganizes mirrored local evidence into translation, rotation, and gripper modes, and uses conditional pair guidance to favor locally better continuations. Experiments on flow-based manipulation benchmarks show that PAWN improves adaptation efficiency, task performance, and OOD robustness over critic-free baselines, while remaining competitive with strong critic-based methods in benchmark evaluations.

Keywords: Flow-Matching, Reinforcement Learning, Vision-Language-Action

1 Introduction

Generative policies based on denoising diffusion [1] and flow matching [2, 3] have become a powerful paradigm for high-dimensional action generation [4, 5, 6, 7]. Because actions are produced through iterative stochastic or deterministic sampling rather than an explicit tractable policy density, the action likelihoods required by standard policy-ratio objectives are often unavailable or computationally intractable in practice. This issue is particularly relevant in robot manipulation and vision-language-action (VLA) systems, where diffusion- and flow-based action models have shown strong promise for visuomotor control and generalist robot policies [8, 9, 10]. Thus, online reinforcement learning for generative robot policies has become an increasingly important direction [11].

A recent line of work suggests a new route for this difficulty. Based on Negative-aware Fine-Tuning (NFT) [12], which models negative answers through an implicit negative policy and uses positive and negative generations for supervised self-improvement of verification-driven LLM training, DiffusionNFT [13] adapts this principle to diffusion models by optimizing the forward process via flow matching and contrasting positive and negative generations to obtain an implicit improvement direction without explicit likelihood estimation. However, *transferring this philosophy from diffusion generation to embodied flow-based policies is far from straightforward*: In flow-based VLAs, a multi-step solver trajectory generates a state-conditioned action chunk; effective online exploration

10th Conference on Robot Learning (CoRL 2026), Austin, Texas, USA.

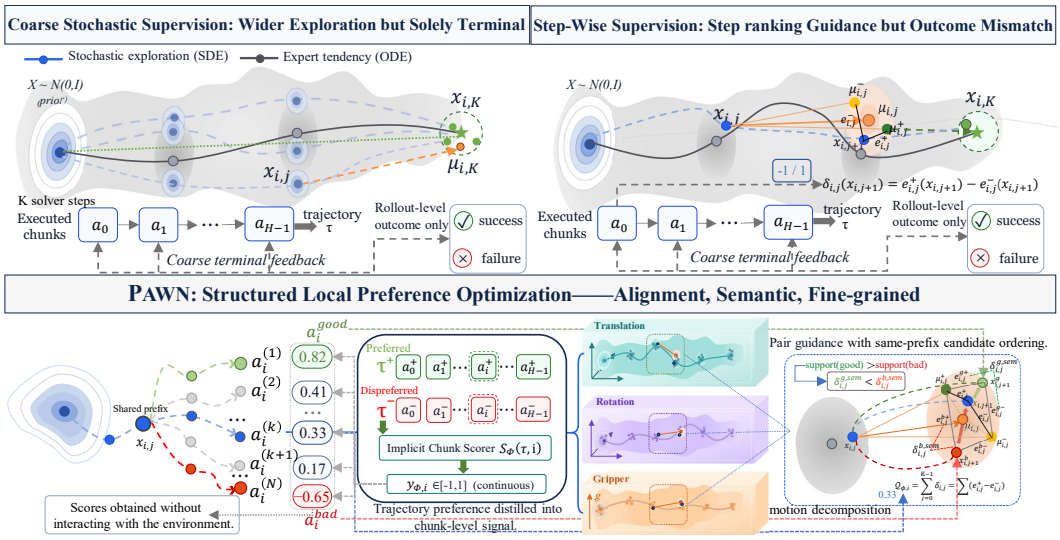


Figure 1: Comparison of likelihood-free local optimization. Coarse stochastic supervision expands exploration but yields a coarse correction that lags behind on-policy drift; Step-wise correction refines denoising updates yet still inherits trajectory-level labels. PAWN converts preferences into chunk scores, decomposes motion semantics, and pair-guides toward better continuations.

often requires stochastic local transitions; and supervision is observed after generated chunks interact with a sequential environment. As a result, once NFT-style local optimization is brought into flow-based manipulation, the object of local supervision becomes substantially more ambiguous.

Recent work has taken an important step toward embodied NFT-style RL by coupling Flow-SDE exploration with step-wise mirrored supervision for flow-based VLAs [14], which widens the exploration space while providing finer local correction. Yet several structural gaps remain. First, a binary trajectory outcome is still too coarse even for an executed action chunk, and becomes more weakly aligned when assigned to intermediate solver states: useful local behaviors in failed rollouts may be suppressed, while unstable or accidental motions in successful rollouts may be reinforced. Second, a single full-space mirrored margin is agnostic to the compositional structure of robot actions, mixing task-relevant evidence with unrelated motion directions. Third, the update remains tied to the behavior-sampled continuation, without an explicit local comparison that indicates which nearby denoising direction leads to a better action. These limitations suggest that embodied NFT-style RL requires a more structured form of local preference optimization.

In this paper, we present **P**reference-**AW**are Semantic **N**FT (**PAWN**), a structured NFT-style online RL framework for flow-based manipulation policies. Our central idea is to replace coarse rollout-level correction with local preference optimization that is aligned with both the solver dynamics and the physical structure of robot actions: We first derive an implicit chunk-level preference signal from trajectory preferences: by lifting solver-step density comparisons to chunk-path likelihood ratios, a Bradley–Terry preference objective induces a continuous local quality score expressed as accumulated solver-level energy gaps, without requiring explicit action likelihoods or manually annotated chunk rewards. This score then serves as the common local credit signal for the rest of the framework: (i) To make the resulting updates physically meaningful, we reorganize mirrored NFT evidence into subspaces of the normalized action space, separating translation, rotation, and end-effector actuation with optional axis-level refinement; (ii) Rather than only updating around the behavior-sampled continuation, we branch multiple suffix chains from the same solver prefix, rank these suffix paths using the learned local preference signal, and impose a pairwise constraint that explicitly guides the current policy toward a more optimal action manifold. Together, these steps turn likelihood-free NFT-style optimization into a chunk-aware, motion-aware, and continuation-aware framework for embodied flow policies (Figure 1).

We validate PAWN through extensive experiments on LIBERO [15] and ManiSkill [16] benchmarks. It consistently improves both optimization efficiency and final performance: On LIBERO, PAWN attains **94.8/97.9%** average success for $\pi_0/\pi_{0.5}$, outperforming SFT by **37.2/20.8 pp**, and reaches its own-plateau T_{90} at least **5** steps earlier than critic-free RL baselines; π_0 gains **2.6 pp** over π-StepNFT on ManiSkill OOD. In summary, we make the following contributions: 1) we introduce PAWN, an NFT-style online RL framework for flow-based VLAs that reformulates online adaptation as structured local preference optimization over solver paths, action chunks, and robot motion subspaces; 2) we derive an implicit chunk-level preference signal from trajectory preferences by lifting solver-step density ratios to chunk-path likelihood ratios, enabling fine-grained local supervision without explicit action likelihoods or manually annotated chunk rewards; 3) we introduce motion-mode probing and conditional pair guidance, which respectively organize mirrored NFT evidence in physically meaningful motion coordinates and provide explicit same-prefix ranking toward higher-quality local continuations.

2 Related Works

2.1 Implicit Flow-Space Optimization for Generative Models and Policies

Early work on generative model optimization primarily relied on feedback, preference, or reward signals defined on final outputs [17, 18, 19, 20, 21, 22, 23, 24, 25, 26]. Recent efforts have shifted toward optimization within the model's internal generative dynamics, including direct optimization in score or flow space [27, 28, 29, 30] and finer-grained optimization along denoising trajectories [31]. A parallel line of work leverages intermediate denoising signals to improve credit assignment and alignment efficiency [32, 33, 34]. LPO [35] learns a noise-aware latent reward model and performs preference optimization directly in noisy space. More recently, GLASS Flows [36] improves reward alignment through stochastic transition sampling in flow dynamics while preserving the efficiency of ODE-style inference, whereas DiffusionNFT [13] performs policy improvement directly in the forward process via flow matching and positive–negative trajectory contrast, thereby avoiding explicit likelihood estimation. Nonetheless, most of the methods driven by reward or preference supervision have limitations in the mismatch between output-level feedback and internal generative process.

2.2 Online RL for Improving VLA Robustness and Exploration

Online reinforcement learning [37] is increasingly becoming a key complement to VLA [38] post-training. One line of work applies on-policy optimization directly to auto-regressive policies [39, 40], often relying on process rewards and careful systems design to mitigate sparse success signals and unstable training [41, 42, 43]. Another emphasizes stabilization, combining localized RL updates during exploration with supervised consolidation of successful online trajectories [44, 45]. For generative policies [8, 9, 4, 46], recent studies have been further split into likelihood-based policy-gradient methods [47, 48, 11] and likelihood-free [49] optimization schemes. Besides, world-model-based rollouts [50] and latent-representation-based reward shaping [51] have also emerged as promising directions to improve real-world sample efficiency. Additionally, π-StepNFT [14] attempted to apply the concepts of DiffusionNFT to online RL for VLAs by introducing SDE-based sampling and shifting supervision to the next denoising step, but such methods' optimization signal lacks explicit guidance for the object updating direction. However, many methods still operate on monolithic high-dimensional action chunks, rather than explicitly leveraging the latent multimodal manifold structure and semantic regularities of the action space to enable decoupled and fine-grained policy refinement.

3 Preliminaries

We first introduce the notation for flow-based embodied policies, emphasizing the two coupled time scales induced by action generation. We then briefly review the NFT view from general generative modeling, which provides a likelihood-free optimization perspective that motivates our method.

3.1 Flow-Based Embodied Policy in an MDP

We consider embodied manipulation as a finite-horizon Markov decision process (MDP). A rollout is denoted by $\tau = \{(s_i, a_i)\}_{i=0}^{H-1}$, where i indexes replanning decisions, H counts action chunks, s_i is the robot-environment state, and $a_i = [u_{i,0}, \ldots, u_{i,L-1}] \in \mathbb{R}^{L\times 7}$ is an action chunk of L physical controls indexed by ℓ. In a flow-based policy, an action chunk is not produced in a single feed-forward pass. Instead, conditioned on the control context c_i (visual observations, language instruction, and proprioception), the policy generates a latent action path through a K-step solver rollout, $\zeta_i = (x_{i,0}, \ldots, x_{i,K})$, with $a_i = x_{i,K}$ and schedule $1 = \rho_0 > \cdots > \rho_K = 0$. Each $x_{i,j} \in \mathbb{R}^{L\times 7}$ is an intermediate solver state, with j indexing denoising. Replanning and within-chunk denoising form two coupled time scales; outcome labels come from the physical rollout.

The policy is parameterized by a time-dependent vector field $v_\theta(x, \rho, c_i)$. Under deterministic sampling, the latent path is generated by integrating this field from $\rho = 1$ to $\rho = 0$. For online RL, however, purely deterministic rollouts provide limited exploration, so during training we adopt stochastic solver transitions. For a local step from ρ to $\rho^- = \rho - \Delta\rho$, we write the Euler–Maruyama update as

$$x^{\rho^-} = x^\rho + \Big[v_\theta(x^\rho, \rho, c_i) + \frac{\sigma_\rho^2}{2\rho}\big(x^\rho + (1-\rho)v_\theta(x^\rho, \rho, c_i)\big)\Big](-\Delta\rho) + \sigma_\rho\sqrt{\Delta\rho}\,\epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}), \tag{1}$$

which induces a Gaussian local transition $p_{\theta,\rho}(x^{\rho^-} \mid x^\rho, c_i) = \mathcal{N}(\mu_{\theta,\rho}(x^\rho, c_i), \Sigma_\rho)$. Importantly, its mean is an affine function of the velocity field:

$$\mu_{\theta,\rho}(x^\rho, c_i) = M_\rho(\rho)\, v_\theta(x^\rho, \rho, c_i) + W_\rho(x^\rho, \rho), \tag{2}$$

where W_ρ and M_ρ are coefficients determined by the noise schedule and the discretization step. The explicit expressions are given in Appendix D.1. In the sequel, credit will be assigned over the inner solver path ζ_i, rather than only over the executed terminal action a_i.

3.2 Negative-Aware Fine-Tuning for Likelihood-Free Optimization

The NFT paradigm avoids explicit policy-ratio computation and instead defines policy improvement through a contrast between positive and negative feedback (Appendix B.1).

Given a reference policy v^{ref} and a trainable policy v_θ, NFT constructs two implicit branches around the reference policy:

$$v_\theta^+ = v^{\text{ref}} + \beta\,\Delta v, \qquad v_\theta^- = v^{\text{ref}} - \beta\,\Delta v, \tag{3}$$

where $\Delta v := v_\theta - v^{\text{ref}}$, and $\beta > 0$ controls the update strength. This view encourages the policy to move toward directions supported by positive samples while being repelled from directions associated with negative ones, yielding a likelihood-free optimization principle for generative policies.

4 Methodology

4.1 From Local Solver Densities to an Implicit Chunk Scorer

We first derive a chunk-level scorer from the stochastic solver dynamics in Sec. 3.1. The key idea is to avoid explicit action-likelihood evaluation by comparing tractable local solver densities, and then lift these local comparisons from individual solver steps to the whole action-generation path (motivation in Appendix B.2).

Consider the local transition $x_{i,j} \to x_{i,j+1}$ in chunk i at solver time ρ_j. For any model or branch ω, the Euler–Maruyama step induces a Gaussian transition $p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i) = \mathcal{N}\big(\mu_{\omega,\rho_j}(x_{i,j}, c_i), \Sigma_{\rho_j}\big)$, where μ_{ω,ρ_j} follows the affine velocity-to-mean map in Eq. (2). We define a unified local transition energy for any target z: $e_{\omega,i,j}(z) := \big\|z - \mu_{\omega,\rho_j}(x_{i,j}, c_i)\big\|^2_{\Sigma_{\rho_j}^{-1}}$. For two models ω and $\bar{\omega}$ sharing the same covariance, their local log-density ratio on the observed transition is

$$\log \frac{p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)}{p_{\bar{\omega},\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)} = -\frac{1}{2}\left[e_{\omega,i,j}(x_{i,j+1}) - e_{\bar{\omega},i,j}(x_{i,j+1})\right]. \tag{4}$$

This follows directly from the Gaussian log-density with shared covariance; the derivation is given in Appendix D.2.

We next lift this local identity to the chunk path $\zeta_i = (x_{i,0}, \dots, x_{i,K})$, whose terminal output is the executed action chunk $a_i = x_{i,K}$. Given $x_{i,0}$ and context c_i, define the path likelihood

$$\mathcal{Q}_\omega(\zeta_i \mid c_i) = \prod_{j=0}^{K-1} p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i). \tag{5}$$

This is a solver-path likelihood rather than the marginal likelihood of the terminal action. At the rollout level, environment transitions do not depend on the compared solver model and cancel in model ratios; hence trajectory log-ratios reduce to additive sums of chunk-path log-ratios. We provide the full path-lifting derivation in Appendix D.3.

At outer iteration m, we define the raw implicit score of chunk i by comparing the trainable scorer $\phi^{(m)}$ with the previous rollout snapshot b_{m-1}:

$$u_{\phi,i}^{(m)} := \beta \log \frac{\mathcal{Q}_{\phi^{(m)}}(\zeta_i \mid c_i)}{\mathcal{Q}_{b_{m-1}}(\zeta_i \mid c_i)} = -\frac{\beta}{2} \sum_{j=0}^{K-1} \left[e_{\phi^{(m)},i,j}(x_{i,j+1}) - e_{b_{m-1},i,j}(x_{i,j+1}) \right]. \tag{6}$$

Thus $u_{\phi,i}^{(m)}$ is a pathwise density-ratio score accumulated over the local solver transitions that generate chunk a_i. And the scorer is learned from trajectory preferences: Given a preferred pair (τ^+, τ^-) collected under the same task condition and initial state, we optimize

$$\mathcal{L}_{\mathrm{sc}}(\phi) = -\mathbb{E}_{(\tau^+,\tau^-)} \left[\log \sigma \left(\sum_i u_{\phi,i}^+ - \sum_i u_{\phi,i}^- \right) \right]. \tag{7}$$

This Bradley–Terry objective makes the sum of chunk scores agree with the observed rollout preference. We then robustly normalize $u_{\phi,i}$ into a bounded local label $y_{\phi,i} \in [-1, 1]$, and use an outcome-consistency gate g_i to suppress unreliable chunks (details are given in Appendix B.3).

Finally, we define how this learned chunk signal enters actor optimization. Around the rollout snapshot b_m, let $\Delta v_{i,j} = v_\theta(x_{i,j}, \rho_j, c_i) - v_{b_m}(x_{i,j}, \rho_j, c_i)$, $v_{i,j}^\pm = v_{b_m}(x_{i,j}, \rho_j, c_i) \pm \eta_j \Delta v_{i,j}$, and let $\mu_{i,j}^\pm$ be the corresponding Gaussian means. For any local target z, define the actor mirrored gap $\delta_{i,j}(z) := e_{+,i,j}(z) - e_{-,i,j}(z)$. For the observed transition, write $\delta_{i,j} := \delta_{i,j}(x_{i,j+1})$ and $\bar{\delta}_i = K^{-1} \sum_j \delta_{i,j}$. The base dual-credit actor losses are

$$\mathcal{L}_{\mathrm{traj}} = \frac{1}{H} \sum_i \mathrm{softplus}\left(\frac{1}{2} y_\tau \, \delta_{i,j_i} \right), \quad \mathcal{L}_{\mathrm{chunk}} = \frac{1}{H} \sum_i g_i \, \mathrm{softplus}\left(\frac{1}{2} y_{\phi,i} \, \bar{\delta}_i \right). \tag{8}$$

where $j_i \sim \mathcal{U}\{0, \dots, K-1\}$. The first term preserves rollout-level outcome supervision, while the second injects the learned chunk-level preference into the same mirrored density-ratio geometry.

Nevertheless, in the actual training process, directly deriving an implicit chunk scorer in this manner still presents challenges; to address this, in Appendix C, we propose an efficient training method.

4.2 Motion-Mode Probing of Local NFT Margins

The mirrored gap in Eq. (8) treats the action chunk as a homogeneous vector. This is suboptimal for manipulation, where translation, rotation, and end-effector actuation play different physical roles and may contribute unequally to task progress. We therefore reorganize local NFT evidence in a motion-mode basis, so that the actor update can identify not only whether a local transition is preferred, but also which motion mode supports that preference.

Let $\mathcal{H} = \{\mathrm{tr}, \mathrm{rot}, \mathrm{gr}\}$ denote translation, rotation, and gripper/end-effector blocks. For each physical control $u_{i,\ell} = [x, y, z, r_x, r_y, r_z, g]^\top$, these blocks are implemented after the checkpoint's per-dimension normalization by coordinate projectors $P_{\mathrm{tr}}, P_{\mathrm{rot}}, P_{\mathrm{gr}}$. We further use axis-level projectors $\mathcal{A}_{\mathrm{tr}} = \{x, y, z\}$ and $\mathcal{A}_{\mathrm{rot}} = \{r_x, r_y, r_z\}$, while keeping the gripper block atomic because it behaves more like a low-dimensional actuation mode than a smoothly decomposable spatial motion.

For chunk i and solver step j, we probe the local update direction by mirroring it only inside one semantic subspace:

$$v_{i,j}^{h,\pm} = v_{b_m}(x_{i,j}, \rho_j, c_i) \pm \eta_j P_h \Delta v_{i,j}, \qquad h \in \mathcal{H}.$$

Using the same transition energy $e_{\omega,i,j}$, this gives a block-wise mirrored gap $\delta_{i,j,h}^{\mathrm{blk}}$. Applying the same construction to an axis projector P_d gives an axis-wise gap $\delta_{i,j,d}^{\mathrm{ax}}$. Under a block-compatible covariance metric, the full-space gap admits the decomposition

$$\delta_{i,j}^{\mathrm{full}} = \sum_{h\in\mathcal{H}} \delta_{i,j,h}^{\mathrm{blk}}, \qquad \delta_{i,j,h}^{\mathrm{blk}} = \sum_{d\in\mathcal{A}_h} \delta_{i,j,d}^{\mathrm{ax}}. \tag{9}$$

The derivation follows from the affine mean map and projector decomposition, and is given in Appendix D.4. In the general case, we use this decomposition as a structured semantic surrogate rather than assuming dynamic independence among blocks.

We then use the learned chunk signal to select reliable semantic evidence. Let $\tilde{y}_i = g_i y_{\phi,i}$ be the gated local supervision signal. Its agreement with block and axis probe margins defines nonnegative weights $\omega_{i,h}$ and $\omega_{i,d|h}$, detailed in Appendix D.5. The resulting per-step semantic gap and its chunk-aggregated form are:

$$\delta_{i,j}^{\mathrm{sem}} = \sum_{h\in\mathcal{H}} \omega_{i,h} \left[(1-\lambda_{\mathrm{ax}})\delta_{i,j,h}^{\mathrm{blk}} + \lambda_{\mathrm{ax}} \sum_{d\in\mathcal{A}_h} \omega_{i,d|h}\delta_{i,j,d}^{\mathrm{ax}} \right], \quad \bar{\delta}_i^{\mathrm{sem}} = \frac{1}{K}\sum_{j=0}^{K-1} \delta_{i,j}^{\mathrm{sem}}. \tag{10}$$

With nonnegative weights totaling at most one, $\bar{\delta}_i^{\mathrm{sem}}$ is bounded in magnitude by the probe margins and preserves the preference sign when the active block/axis evidence is aligned with $\tilde{y}_i$; we prove these properties in Appendix D.5.

Finally, semantic probing replaces the full-space gaps in Eq. (8) without changing the dual-credit structure: $\mathcal{L}_{\mathrm{traj}}^{\mathrm{sem}} = \mathcal{L}_{\mathrm{traj}}\big[\delta_{i,j_i} \leftarrow \delta_{i,j_i}^{\mathrm{sem}}\big], \mathcal{L}_{\mathrm{chunk}}^{\mathrm{sem}} = \mathcal{L}_{\mathrm{chunk}}\big[\bar{\delta}_i \leftarrow \bar{\delta}_i^{\mathrm{sem}}\big]$. Thus, motion-mode probing restructures the same mirrored local evidence into physically meaningful motion subspaces.

4.3 Conditional Pair Guidance over Candidate Suffix Chains

Despite the aforementioned methods, conventional NFT approaches still suffer from certain shortcomings. For example, the actor is still optimized around the continuation sampled by the rollout behavior. This gives a local preference sign but not an explicit nearby target that indicates where the denoising path should move next; as a result, updates may follow suboptimal directions before reaching high-quality action regions. Towards this, we introduce *conditional pair guidance*.

For chunk i, we sample a solver index $j \sim \mathcal{U}\{0,\dots,K-1\}$ and fix the shared prefix $x_{i,j}$. Conditioned on the same state s_i, context c_i, and prefix position j, we resample n suffix chains from the behavior snapshot b_m:

$$\zeta_{i,j:K}^{(r)} = (x_{i,j}, x_{i,j+1}^{(r)}, \dots, x_{i,K}^{(r)}), \qquad a_i^{(r)} = x_{i,K}^{(r)}, \quad r = 1,\dots,n.$$

We score these internally sampled suffixes with the frozen scorer: $s_i^{(r)} = S_\phi(\zeta_{i,j:K}^{(r)} \mid s_i, c_i)$, using the pathwise energy-gap sum in Eq. (6) over the suffix transitions. The shared prefix cancels in score differences. The highest- and lowest-scoring suffixes define a pair when their score gap exceeds δ_{pair}. This ranking uses solver traces and requires no additional environment execution.

Let $x_{i,j+1}^{g}$ and $x_{i,j+1}^{b}$ be the first successor states of the selected good and bad suffixes. Reusing the semantic mirrored-gap operator from Sec. 4.2, we evaluate

$$\delta_{i,j}^{g,\mathrm{sem}} := \delta_{i,j}^{\mathrm{sem}}(x_{i,j+1}^{g}), \qquad \delta_{i,j}^{b,\mathrm{sem}} := \delta_{i,j}^{\mathrm{sem}}(x_{i,j+1}^{b}).$$

A smaller mirrored gap indicates stronger positive-vs-negative support. Therefore, the actor should satisfy $\delta_{i,j}^{g,\mathrm{sem}} < \delta_{i,j}^{b,\mathrm{sem}}$, meaning that the good suffix is locally more supported than the bad suffix under the same prefix. We enforce this ordering with

$$L_{\mathrm{pair},i} = w_{\mathrm{pair},i}\,\mathrm{softplus}\left(\frac{\kappa}{2}\big(\delta_{i,j}^{g,\mathrm{sem}} - \delta_{i,j}^{b,\mathrm{sem}}\big)\right), \tag{11}$$

where $\kappa > 0$ controls ranking sharpness and $w_{\text{pair},i}$ is a confidence weight derived from the scorer gap. The confidence weighting is detailed in Appendix B.4, and the Bradley–Terry interpretation of Eq. (11) is derived in Appendix D.6. Aggregating valid pairs $\mathcal{P}$, we define $\mathcal{L}_{\text{pair}} = |\mathcal{P}|^{-1} \sum_{i \in \mathcal{P}} L_{\text{pair},i}$. The final actor objective is

$$\mathcal{L}_{\text{total}} = \mathcal{L}^{\text{sem}}_{\text{traj}} + \lambda_\phi \mathcal{L}^{\text{sem}}_{\text{chunk}} + \lambda_{\text{pair}} \mathcal{L}_{\text{pair}} + \lambda_{\text{TR}} \mathbb{E}_{i,j} \|\Delta v_{i,j}\|_2^2. \tag{12}$$

Thus, the final update combines rollout-level outcome supervision, learned chunk-level credit, motion-mode structure, and same-prefix pairwise preference. In effect, the actor is not only told whether a sampled transition is good, but is also guided toward locally better action continuations within the semantic motion space.

5 Experiments

Setup and metrics. We evaluate PAWN on LIBERO [15] Spatial/Object/Goal/Long and ManiSkill [16] manipulation benchmarks, following the same online RL protocol as prior flow-policy adaptation baselines. We use the OpenPI π_0 and $\pi_{0.5}$ initializations, freeze the vision-language backbone, and update only the flow-based action expert during online adaptation. We compare against PPO [37], GRPO [52] (instantiated with the Flow-SDE recipe for flow-based VLA policies, following the π_{RL} setting [47]) and π-StepNFT under the same rollout budget and evaluation frequency, reporting final success rate (SR, %) and T_{90}, the first training step at which the smoothed success curve reaches 90% of its final plateau. The details of the implementation and the hyperparameters are provided in Appendix E. Real-world results are reported in Sec. 5.2, with full details in Appendix F. T_{90} measures convergence to each method's own plateau and is read jointly with final SR. For comparisons with SDE-PPO, we also report SR at a common budget $B = 400$ epochs (SR@B) and $\text{nAUC}(B) = B^{-1} \int_0^B \text{SR}(b)\, db$, with SR on a $[0, 1]$ scale (Table 3). Table 1 uses the full 400-epoch protocol; Figure 2 shows selected π_0 Object/Long runs over steps 0–200.

5.1 Simulation Evaluation

Init.	Method	Spatial	Object	Goal	Long	Avg. ↑	T_{90} ↓
π_0	SFT	65.3	64.4	49.8	51.2	57.6	-
	SDE-PPO	**98.4**	**99.4**	**96.2**	90.2	**96.0**	51
	SDE-GRPO	97.8	97.8	83.2	81.4	90.0	44
	π-StepNFT	93.5	98.0	83.7	86.7	90.5	44
	PAWN	97.2	98.9	92.8	**90.3**	94.8	**39**
$\pi_{0.5}$	SFT	84.6	95.4	84.6	43.9	77.1	–
	SDE-PPO	**99.6**	**100.0**	98.8	93.0	**97.9**	47
	SDE-GRPO	97.4	99.8	91.2	77.6	91.5	41
	π-StepNFT	97.8	**100.0**	98.2	79.8	94.0	43
	PAWN	98.7	**100.0**	**99.2**	**93.4**	**97.9**	**36**

Table 1: Success rate (%) and adaptation speed T_{90} on LIBERO. Using OpenPI models after few-shot SFT.

LIBERO results. Table 1 shows that PAWN achieves higher average SR than SFT and strong critic-free baselines across initializations, with lower own-plateau T_{90} than the critic-free RL baselines. It remains competitive in final SR with critic-based SDE-PPO and achieves higher common-budget nAUC (Table 3), supporting more efficient adaptation. These results reflect two coupled effects: (1) stochastic Flow-SDE rollouts expand exploration beyond the imitation manifold, (2) mirrored updates and better same-prefix continuations in translation, rotation, and gripper subspaces make that exploration more trainable. Object and Goal SR reach 100% and 99.2%, respectively, reflecting improved object- and goal-conditioned manipulation. Long tasks particularly benefit from chunk-level preference guidance that aligns delayed outcomes with local action quality. The earlier rise and higher plateau of the return curve in Figure 2 are consistent with these success gains.

ManiSkill generalization. Table 2 and the left bottom of Figure 2 evaluate whether PAWN improves robustness under distribution shifts. Outcome-grounded, critic-free supervision in π-StepNFT avoids dependence on a learned value function, a potential source of sensitivity to nuisance visual or language features. PAWN's implicit chunk scorer provides quality signals that are more aligned with action-level progress, helping the policy focus on reusable manipulation patterns. Meanwhile, motion-mode probing guides updates toward the motion modes that remain meaningful across unseen scenes and incorporates the pair-guidance mechanism to accelerate policy adaptation. It improves π_0 IND/OOD SR by 3.2/2.6 pp over π-StepNFT and $\pi_{0.5}$ OOD SR by 2.1 pp, supporting the benefits of motion-aware supervision.

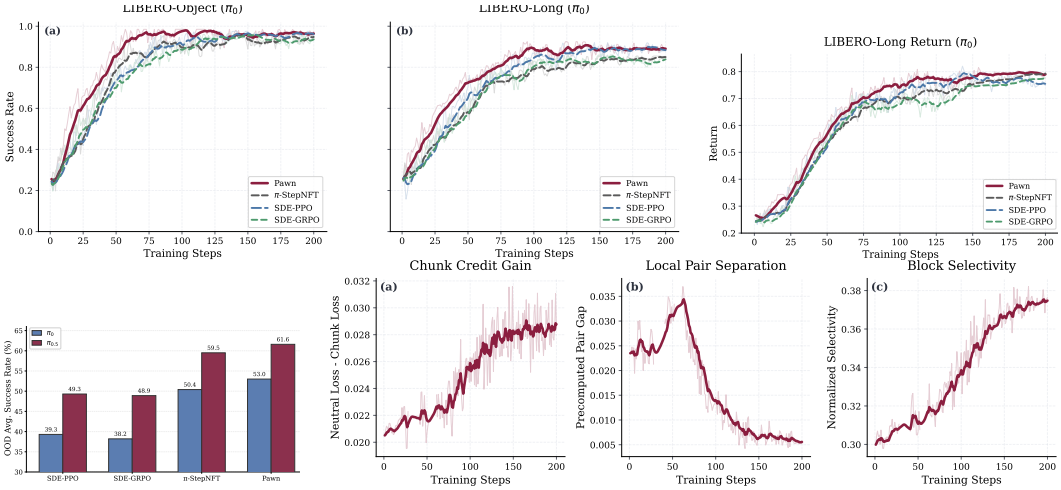


Figure 2: Training curves and mechanism diagnostics. Top: PAWN improves LIBERO success rate and T_{90}, also features a faster upward and consistently higher return curve. Bottom: (left) It also improves ManiSkill OOD adaptation, (right) while diagnostics show increasing chunk-credit gain, explicit local pair separation, and stronger block selectivity, linking chunk supervision, candidate ranking, and motion selectivity to the observed training dynamics.

Mechanistic evidence. The components of PAWN form a hierarchical local-preference framework. Figure 2 tracks three diagnostics: (a) *Chunk Credit Gain*, measured as the loss reduction relative to neutral chunk supervision, rises throughout training, indicating increasingly useful local supervision; (b) *Local Pair Separation* first grows and then declines as the policy improves, suggesting that pair guidance is most informative while nearby good/bad suffixes remain distinguishable; and (c) *Block Selectivity* rises as semantic probing concentrates credit on dominant motion modes. The confidence weight emphasizes pairs with larger score gaps, connecting local pair separation to the actor update. Together, these trends support the intended progression from trajectory preferences to chunk credit, local continuation ranking, and structured motion evidence. Held-out task-progress validation further shows that scorer gaps track short-horizon predicate progress and identify better downstream continuations (Appendix C.3), while Table 3 measures component contributions.

Init.	Method	IND	OOD			
			Vision	Sem.	Exec.	Avg. ↑
π_0	π-StepNFT	79.2	69.1	49.1	33.1	50.4
	PAWN	**82.4**	**72.0**	**51.6**	**35.4**	**53.0**
$\pi_{0.5}$	π-StepNFT	85.4	76.9	56.6	45.1	59.5
	PAWN	**87.6**	**78.3**	**58.9**	**47.5**	**61.6**

Table 2: Success rates (%) on ManiSkill in in-distribution (IND) and out-of-distribution (OOD).

Component Ablations Table 3 evaluates the implicit chunk scorer (C), motion-mode probing (M), and pair guidance (P). Both M/P depend on C: its gated labels weight motion evidence, while the scorer ranks suffixes; w/o M uses full-space mirrored gaps. The w/o M & P row retains C. Each seed uses the same 500 held-out episodes per suite (50 resets $\times$ 10 subtasks). Adding C improves mean Goal/Long SR by 2.9 pp and nAUC by 0.032, supporting the value of localizing trajectory supervision. Given C, adding M or P separately yields 2.1/1.4 pp, with nAUC gains of 0.009/0.004. These gains are consistent with M selecting useful control components and P identifying better continuations. To-

Method	Goal SR	Long SR	Avg. nAUC
SDE-PPO	$\mathbf{95.8 \pm 0.6}$	89.9 ± 1.1	0.685 ± 0.033
π-StepNFT	83.5 ± 0.9	86.3 ± 1.3	0.763 ± 0.038
w/o M & P	87.5 ± 1.1	88.1 ± 0.8	0.795 ± 0.043
w/o P	90.2 ± 0.7	89.6 ± 0.7	0.804 ± 0.046
w/o M	89.4 ± 0.8	88.9 ± 0.9	0.799 ± 0.039
PAWN	92.3 ± 1.0	$\mathbf{90.2 \pm 0.8}$	$\mathbf{0.819 \pm 0.041}$

Table 3: Baselines and ablations with π_0 on LIBERO Goal/Long. Mean±std over three seeds; SR (%) at common budget B, and nAUC averaged over both suites.

gether, $M + P$ add 3.5 pp and 0.024 nAUC over C alone, supporting complementary refinement of local updates. The nAUC gain from P rises to 0.015 when M is present, suggesting that structured motion credit helps the actor use ranked continuations more effectively. Full PAWN improves Goal/Long SR by 8.8/3.9 pp and nAUC by 0.056 over π-StepNFT. Against SDE-PPO, it achieves competitive SR and higher nAUC (0.819 vs. 0.685), indicating stronger performance across the training budget.

5.2 Real-World Adaptation and Deployment

Beyond simulation, Figure 3 evaluates PAWN in a real-world batch adaptation setting. Starting from a few-shot ($\pi_{0.5}$) policy, we collect real demonstrations, autonomous successes/failures, and human interventions, then perform offline PAWN adaptation. It suggests strong real-world deployment robustness under the same real-data budget.

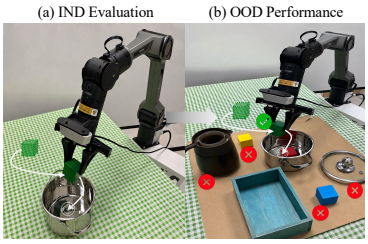


Figure 3: Real world evaluation of PAWN. It executes successful rollouts in clean and cluttered tabletop scenes.

We use a Piper COBOT MAGIC arm for block-into-pot (T1) and lid-on-pot (T2) tasks. Each task uses a fixed batch of 20 demonstrations, 30 autonomous rollouts, and 10 intervention trajectories. All three methods share the few-shot $\pi_{0.5}$ initialization, frozen VLM, action-expert update budget, and evaluation protocol. Offline π-StepNFT keeps the same flow-matching, intervention, and trust-region terms, replacing PAWN's chunk/motion/pair terms with step-wise mirroring on identical stored or reconstructed transitions.

Across 20 held-out trials per task and setting, PAWN achieves 28/40 OOD successes, compared with 25/40 for π-StepNFT and 21/40 for Real SFT, with the smallest clean-to-OOD drop (Table 4). Under clutter and pose variation, both tasks benefit: T1 requires reliable grasping and release into the pot, while T2 demands precise lid–pot alignment. These gains are consistent with chunk-level credit and motion-mode probing refining local translation, rotation, and gripper decisions, while pair guidance favors better continuations. This supports more effective use of sparse outcome feedback under matched data budgets. Appendix F provides details and clean/OOD sequences (Figures 8 and 9).

Method	T1 C/O	T2 C/O	Total C/O	Drop
Real SFT	18/13	12/8	30/21	22.5
π-StepNFT	17/15	14/10	31/25	15.0
PAWN	18/16	14/12	32/28	10.0

Table 4: Real-world Evaluation. C/O: clean/OOD successes out of 20 per task, 40 in total. Drop: clean-to-OOD decrease in pp.

6 Limitations

Summary We presented PAWN, a unified NFT-style online RL framework for flow-based manipulation policies. Our method employs structured local preference optimization by deriving implicit chunk-level scores from trajectory preferences, organizing mirrored comparisons in decoupled motion subspaces, and guiding updates through same-prefix good and bad continuations. Experiments demonstrate improved optimization efficiency and final performance over strong baselines, providing a new paradigm for online RL algorithms for robotic policies.

Our method also has some limitations. It assumes access to additional rollout data and meaningful trajectory preferences, so performance may degrade when exploration produces little success and failure diversity or when the learned chunk scorer receives poorly separated preference pairs. The motion-mode decomposition also relies on a canonical action parameterization with identifiable translation, rotation, and end-effector components, which may require adaptation for highly coupled dexterous hands or non-standard robot embodiments. While we include real-world deployment experiments with offline real-batch adaptation, we do not yet conduct autonomous real-world online RL on hardware. Future work should study safe on-robot policy updates under strict safety constraints, richer sensory feedback, and longer-horizon contact-rich manipulation tasks.

References

[1] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[2] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[3] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.

[4] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[5] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.

[6] S. Ye and M. C. Gombolay. Efficient trajectory forecasting and generation with conditional flow matching. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2816–2823. IEEE, 2024.

[7] S. Jiang, X. Fang, N. Roy, T. Lozano-Pérez, L. P. Kaelbling, and S. Ancha. Streaming flow policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories. In *9th Annual Conference on Robot Learning, CoRL 2025*, Seoul, Korea, September 2025. URL https://openreview.net/forum?id=jnpILGz9gQ.

[8] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[9] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[10] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. RDT-1B: a diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations*, volume 2025, pages 29982–30009, 2025.

[11] T. Zhang, C. Yu, S. Su, and Y. Wang. ReinFlow: Fine-tuning flow matching policy with online reinforcement learning. *arXiv preprint arXiv:2505.22094*, 2025.

[12] H. Chen, K. Zheng, Q. Zhang, G. Cui, Y. Cui, H. Ye, T.-Y. Lin, M.-Y. Liu, J. Zhu, and H. Wang. NFT: Bridging supervised learning and reinforcement learning in math reasoning. In *International Conference on Learning Representations (ICLR)*, 2026.

[13] K. Zheng, H. Chen, H. Ye, H. Wang, Q. Zhang, K. Jiang, H. Su, S. Ermon, J. Zhu, and M.-Y. Liu. DiffusionNFT: Online diffusion reinforcement with forward process. *arXiv preprint arXiv:2509.16117*, 2025.

[14] S. Wang, X. Wang, Z. Zhu, M. Pei, X. Cui, C. Deng, J. Zhao, G. Huang, H. Zhang, and J. Wang. π-StepNFT: Wider space needs finer steps in online RL for flow-based VLAs. *arXiv preprint arXiv:2603.02083*, 2026.

[15] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.

[16] T. Mu, Z. Ling, F. Xiang, D. Yang, X. Li, S. Tao, Z. Huang, Z. Jia, and H. Su. ManiSkill: Generalizable manipulation skill benchmark with large-scale demonstrations. *arXiv preprint arXiv:2107.14483*, 2021.

[17] K. Black, M. Janner, Y. Du, I. Kostrikov, and S. Levine. Training diffusion models with reinforcement learning. *arXiv preprint arXiv:2305.13301*, 2023.

[18] Y. Fan, O. Watkins, Y. Du, H. Liu, M. Ryu, C. Boutilier, P. Abbeel, M. Ghavamzadeh, K. Lee, and K. Lee. DPOK: Reinforcement learning for fine-tuning text-to-image diffusion models. *Advances in Neural Information Processing Systems*, 36:79858–79885, 2023.

[19] K. Clark, P. Vicol, K. Swersky, and D. J. Fleet. Directly fine-tuning diffusion models on differentiable rewards. *arXiv preprint arXiv:2309.17400*, 2023.

[20] X. Wu, Y. Hao, M. Zhang, K. Sun, Z. Huang, G. Song, Y. Liu, and H. Li. Deep reward supervisions for tuning text-to-image diffusion models. In *European Conference on Computer Vision*, pages 108–124. Springer, 2024.

[21] B. Wallace, M. Dang, R. Rafailov, L. Zhou, A. Lou, S. Purushwalkam, S. Ermon, C. Xiong, S. Joty, and N. Naik. Diffusion model alignment using direct preference optimization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8228–8238, 2024.

[22] K. Yang, J. Tao, J. Lyu, C. Ge, J. Chen, W. Shen, X. Zhu, and X. Li. Using human feedback to fine-tune diffusion models without any reward model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8941–8951, 2024.

[23] J. Hong, S. Paul, N. Lee, K. Rasul, J. Thorne, and J. Jeong. Margin-aware preference optimization for aligning diffusion models without reference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 4744–4752, 2026.

[24] D. Zhang, G. Lan, D.-J. Han, W. Yao, X. Pan, H. Zhang, M. Li, P. Chen, D. Yu, C. Brinton, and J. Luo. SePPO: Semi-policy preference optimization for diffusion alignment. *arXiv preprint arXiv:2410.05255v1*, 2024.

[25] X. Zhang, L. Yang, G. Li, Y. Cai, J. Xie, Y. Tang, Y. Yang, M. Wang, and B. Cui. IterComp: Iterative composition-aware feedback learning from model gallery for text-to-image generation. *arXiv preprint arXiv:2410.07171*, 2024.

[26] S. Karthik, H. Coskun, Z. Akata, S. Tulyakov, J. Ren, and A. Kag. Scalable ranked preference optimization for text-to-image generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 18399–18410, 2025.

[27] H. Zhu, T. Xiao, and V. G. Honavar. DSPO: Direct score preference optimization for diffusion model alignment. In *International Conference on Learning Representations*, 2025.

[28] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. *arXiv preprint arXiv:2506.15799*, 2025.

[29] J. Liu, G. Liu, J. Liang, Y. Li, J. Liu, X. Wang, P. Wan, D. Zhang, and W. Ouyang. Flow-GRPO: Training flow matching models via online RL. *arXiv preprint arXiv:2505.05470*, 2025.

[30] J. Fan, S. Shen, C. Cheng, Y. Chen, C. Liang, and G. Liu. Online reward-weighted fine-tuning of flow matching with Wasserstein regularization. In *The Thirteenth International Conference on Learning Representations*, 2025.

[31] D. Kim, S. Lyu, S. W. Kim, and P. H. Seo. Direct diffusion score preference optimization via stepwise contrastive policy-pair supervision. *arXiv preprint arXiv:2512.23426v1*, 2025.

[32] Z. Liang, Y. Yuan, S. Gu, B. Chen, T. Hang, M. Cheng, J. Li, and L. Zheng. Aesthetic post-training diffusion models from generic preferences with step-by-step preference optimization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13199–13208, 2025.

[33] X. Liao, W. Wei, X. Qu, and Y. Cheng. Step-level reward for free in RL-based T2I diffusion model fine-tuning. *arXiv preprint arXiv:2505.19196v1*, 2025.

[34] J. Han, A. Wang, M. Xu, W. Chu, M. Dang, H. Ye, H. Chen, Y. Yue, and S. Ermon. Discrete diffusion trajectory alignment via stepwise decomposition. *arXiv preprint arXiv:2507.04832*, 2025.

[35] T. Zhang, C. Da, K. Ding, H. Yang, K. Jin, Y. Li, T. Gao, D. Zhang, S. Xiang, and C. Pan. Diffusion model as a noise-aware latent reward model for step-level preference optimization. *arXiv preprint arXiv:2502.01051*, 2025.

[36] P. Holderrieth, U. Singer, T. Jaakkola, R. T. Q. Chen, Y. Lipman, and B. Karrer. GLASS flows: Efficient inference for reward alignment of flow and diffusion models. In *The Fourteenth International Conference on Learning Representations*, 2026.

[37] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[38] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

[39] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[40] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.

[41] H. Li, Y. Zuo, J. Yu, Y. Zhang, Z. Yang, K. Zhang, X. Zhu, Y. Zhang, T. Chen, G. Cui, et al. SimpleVLA-RL: Scaling VLA training via reinforcement learning. *arXiv preprint arXiv:2509.09674*, 2025.

[42] G. Lu, W. Guo, C. Zhang, Y. Zhou, H. Jiang, Z. Gao, Y. Tang, and Z. Wang. VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv preprint arXiv:2505.18719*, 2025.

[43] J. Liu, F. Gao, B. Wei, X. Chen, Q. Liao, Y. Wu, C. Yu, and Y. Wang. What can RL bring to VLA generalization? an empirical study. *arXiv preprint arXiv:2505.19789*, 2025.

[44] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao. ConRFT: A reinforced fine-tuning method for VLA models via consistency policy. *arXiv preprint arXiv:2502.05450*, 2025.

[45] Y. Guo, J. Zhang, X. Chen, X. Ji, Y.-J. Wang, Y. Hu, and J. Chen. Improving vision-language-action model with online reinforcement learning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15665–15672. IEEE, 2025.

[46] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[47] K. Chen, Z. Liu, T. Zhang, Z. Guo, S. Xu, H. Lin, H. Zang, Q. Zhang, Z. Yu, G. Fan, et al. π_{RL}: Online RL fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889v1*, 2025.

[48] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz. Diffusion policy policy optimization. *arXiv preprint arXiv:2409.00588*, 2024.

[49] Y. Li, X. Ma, J. Xu, Y. Cui, Z. Cui, Z. Han, L. Huang, T. Kong, Y. Liu, H. Niu, et al. GR-RL: Going dexterous and precise for long-horizon robotic manipulation. *arXiv preprint arXiv:2512.01801*, 2025.

[50] F. Zhu, Z. Yan, Z. Hong, Q. Shou, X. Ma, and S. Guo. WMPO: World model-based policy optimization for vision-language-action models. *arXiv preprint arXiv:2511.09515*, 2025.

[51] S. Fei, S. Wang, L. Ji, A. Li, S. Zhang, L. Liu, J. Hou, J. Gong, X. Zhao, and X. Qiu. SRPO: Self-referential policy optimization for vision-language-action models. *arXiv preprint arXiv:2511.15605*, 2025.

[52] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[53] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[54] L. Yuan, W. Li, H. Chen, G. Cui, N. Ding, K. Zhang, B. Zhou, Z. Liu, and H. Peng. Free process rewards without process labels. *arXiv preprint arXiv:2412.01981*, 2024.

[55] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Alabdulmohsin, M. Tschannen, E. Bugliarello, et al. PaliGemma: A versatile 3B VLM for transfer. *arXiv preprint arXiv:2407.07726*, 2024.

[56] C. Yu, Y. Wang, Z. Guo, H. Lin, S. Xu, H. Zang, Q. Zhang, Y. Wu, C. Zhu, J. Hu, et al. RLinf: Flexible and efficient large-scale reinforcement learning via macro-to-micro flow transformation. *arXiv preprint arXiv:2509.15965*, 2025.

A Notation

Table 5 summarizes the main notation used in the paper.

Table 5: Summary of notation.

Symbol	Meaning
MDP and solver path	
i	Replanning/action-chunk index.
j	Inner solver-step index within an action chunk.
H	Number of action chunks in one rollout.
K	Number of solver steps used to generate one action chunk.
L	Number of physical controls in one action chunk.
ℓ	Physical-control index within an action chunk.
s_i	Robot-environment state or observation at replanning decision i.
c_i	Visual observations, language instruction, and proprioception at step i.
ρ_j	Inner solver time at step j. It is a generation-time variable.
$x_{i,j}$	Solver state in $\mathbb{R}^{L\times 7}$ for chunk i at solver step j.
ζ_i	Solver path of the i-th chunk, $\zeta_i = (x_{i,0}, \ldots, x_{i,K})$.
a_i	Executed action chunk, defined as the terminal solver output $a_i = x_{i,K}$.
τ	Rollout trajectory, typically $\tau = \{(s_i, a_i)\}_{i=0}^{H-1}$.
y_τ	Trajectory-level outcome label, e.g., $y_\tau \in \{-1, +1\}$.
Models and snapshots	
θ	Trainable actor parameters.
v_θ	Current actor velocity field.
b_m	Rollout behavior snapshot at outer iteration m, used for rollout collection and actor mirrored branches.
b_{m-1}	Previous rollout snapshot, used as the reference model for implicit chunk scoring.
ϕ	Trainable implicit chunk scorer, optimized from trajectory preferences rather than manually annotated chunk rewards.
ψ	Generic model index in density-ratio derivations; it may instantiate ϕ, b_{m-1}, or another reference model.
Local transition density and chunk scoring	
$\mu_{\psi,\rho_j}(x_{i,j}, c_i)$	Mean of the local Gaussian transition induced by model ψ.
Σ_{ρ_j}	Shared covariance of the local solver transition at solver time ρ_j.
$y_{\phi,i}$	Normalized chunk-level preference label obtained from $u_{\phi,i}$, bounded in $[-1, 1]$.
Actor mirrored gaps and dual-credit losses	
$v_{i,j}^+, v_{i,j}^-$	Positive and negative mirrored velocity branches.
$\mu_{i,j}^+, \mu_{i,j}^-$	Gaussian means induced by the positive and negative mirrored branches.
$\delta_{i,j}(z)$	Mirrored actor gap for local target z. It corresponds to the positive-vs-negative local margin in NFT-style actor updates.
motion-mode probing	
$\bar{\delta}_{i,h}^{\mathrm{blk}}$	Chunk-aggregated mirrored margin probed in motion block h.
$\bar{\delta}_{i,d}^{\mathrm{ax}}$	Chunk-aggregated mirrored margin probed along axis d.
$\bar{\delta}_i^{\mathrm{sem}}$	Semantic chunk-level mirrored margin.
Conditional pair guidance	
n	Number of candidate suffix chains resampled from the same solver prefix.
$\zeta_{i,j:K}^{(r)}$	The r-th suffix chain resampled from prefix $x_{i,j}$.
$a_i^{(r)}$	Candidate terminal chunk action of suffix r, $a_i^{(r)} = x_{i,K}^{(r)}$.
$s_i^{(r)}$	Pathwise score of candidate suffix $\zeta_{i,j:K}^{(r)}$.
r^+, r^-	Indices of the highest-scoring and lowest-scoring suffixes.
$x_{i,j+1}^g, x_{i,j+1}^b$	First local successor states of the selected good and bad suffix chains.

Continued on next page

Symbol	Meaning
$\delta_{i,j}^{g,\mathrm{sem}}, \delta_{i,j}^{b,\mathrm{sem}}$	Semantic mirrored gaps evaluated on the good and bad local targets.
$w_{\mathrm{pair},i}$	Confidence weight derived from the scorer gap between selected good and bad suffixes.
κ	Pair-guidance ranking sharpness.
$\mathcal{P}$	Set of confident good/bad suffix pairs.

B Details of Methodology

B.1 Background on NFT-Style Likelihood-Free Optimization

We provide a concise overview of the NFT-style optimization paradigm used in this paper. This discussion is intended to clarify the origin of the mirror-branch construction in Sec. 3.2 and the relation between NFT, DiffusionNFT, and flow-based embodied policy optimization.

Negative-aware fine-tuning in LLMs. Negative-aware Fine-Tuning (NFT) [12] was originally proposed for verification-driven LLM training, where a model generates candidate answers and a verifier provides binary outcome labels [53]. Standard supervised self-training typically keeps only correct generations and discards incorrect ones. NFT instead treats the incorrect generations as informative negative feedback: it constructs an implicit negative policy to model the model's own failures, while the target model is optimized on both positive and negative generations. In this sense, NFT is a supervised learning procedure that uses binary feedback to imitate desirable outputs and suppress undesirable ones, without requiring external teacher demonstrations for every step. Its key conceptual contribution is to show that negative samples need not be discarded; they can define an improvement direction that bridges supervised learning and reinforcement learning under verifier feedback.

DiffusionNFT as forward-process likelihood-free RL. DiffusionNFT [13] extends the negative-aware principle to online RL for diffusion and flow models by moving policy optimization from the reverse sampling trajectory to the forward noising process. Given a data-collection policy v^{old}, it first samples clean generations $x_0^{1:K}$ for each condition c, evaluates raw rewards, and converts them into an optimality probability $r(x_0, c) \in [0, 1]$. This induces two implicit reward-conditioned distributions,

$$\pi^+(x_0|c) = \frac{r(x_0,c)}{p_{\pi^{\mathrm{old}}}(o=1|c)}\pi^{\mathrm{old}}(x_0|c), \qquad \pi^-(x_0|c) = \frac{1-r(x_0,c)}{1-p_{\pi^{\mathrm{old}}}(o=1|c)}\pi^{\mathrm{old}}(x_0|c),$$

where π^+ corresponds to positive generations and π^- to negative generations. Since $\pi^+ \succ \pi^{\mathrm{old}} \succ \pi^-$, policy improvement can be viewed as moving the velocity field of v^{old} toward the positive distribution and away from the negative one.

For forward-process training, each clean sample is noised by

$$x_t = \alpha_t x_0 + \sigma_t \epsilon, \qquad v = \dot{\alpha}_t x_0 + \dot{\sigma}_t \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

and DiffusionNFT characterizes a reinforcement guidance direction Δ between the positive, negative, and old velocity fields:

$$\Delta = [1-\alpha(x_t)]\big(v^{\mathrm{old}}(x_t,c,t) - v^-(x_t,c,t)\big) = \alpha(x_t)\big(v^+(x_t,c,t) - v^{\mathrm{old}}(x_t,c,t)\big).$$

Here, $\alpha(x_t) = p_{\pi^{\mathrm{old}}}(o=1 \mid x_t, c, t)$ is the posterior optimality probability, distinct from the noise schedule α_t. A guided improvement target can therefore be written as

$$v^*(x_t,c,t) = v^{\mathrm{old}}(x_t,c,t) + \frac{1}{\beta}\Delta(x_t,c,t).$$

Rather than explicitly training separate positive and negative models, DiffusionNFT uses a single trainable velocity field v_θ to define two implicit branches:

$$v_\theta^+ = (1-\beta)v^{\mathrm{old}} + \beta v_\theta, \qquad v_\theta^- = (1+\beta)v^{\mathrm{old}} - \beta v_\theta.$$

The policy is then updated with a dual forward-process flow-matching objective,

$$\mathcal{L}_{\mathrm{DNFT}}(\theta) = \mathbb{E}_{c,x_0,t}\left[r\,\|v_\theta^+(x_t,c,t) - v\|_2^2 + (1-r)\,\|v_\theta^-(x_t,c,t) - v\|_2^2\right].$$

Under unlimited data and model capacity, the optimum satisfies

$$v_\theta^* = v^{\mathrm{old}} + \frac{2}{\beta}\Delta,$$

which shows that optimizing the supervised dual loss integrates reinforcement guidance into the learned policy. Practically, each online iteration alternates between: (i) collecting clean samples with v^{old}, (ii) assigning normalized optimality probabilities r, (iii) applying forward noising and optimizing the dual objective above, and (iv) softly updating the data-collection policy. This forward-process formulation avoids explicit likelihood estimation, does not require storing reverse sampling trajectories, supports arbitrary black-box solvers, and remains compatible with standard diffusion/flow training pipelines.

Why embodied flow policies require further localization? Directly applying this idea to flow-based robot policies is nontrivial. In image generation, the final generated sample is the object being evaluated. In embodied control, however, the policy generates an action chunk through a short multi-step solver path, executes that action in a sequential environment, and receives task feedback only after physical interaction. Thus, the supervision hierarchy becomes

$$\text{trajectory outcome} \;\rightarrow\; \text{executed action chunks} \;\rightarrow\; \text{intermediate solver states}.$$

Moreover, flow-based VLA policies often use deterministic ODE-style sampling for efficient action generation. Online exploration typically requires stochasticizing this solver into local SDE transitions, but coarse terminal supervision becomes poorly aligned with the intermediate solver states. Recent step-wise NFT method [14] addresses part of this gap by applying mirrored supervision to immediate solver transitions, but they still inherit trajectory-level outcome labels and full-space mirrored comparisons.

Role in our method. Our method follows the same NFT-style principle—replace explicit action likelihoods with local positive-vs-negative comparisons around a reference solver model—but further structures the local supervision for manipulation. First, we convert trajectory-level preferences into implicit chunk-level scores by lifting solver-step density ratios to chunk-path likelihood ratios. Second, we reorganize the mirrored local evidence into translation, rotation, and gripper subspaces. Third, we use the learned chunk-level signal to rank same-prefix candidate continuations, producing an explicit local preference direction. Thus, PAWN can be viewed as a manipulation-specific extension of NFT-style likelihood-free optimization: it keeps the negative-aware mirror geometry, but makes the supervision chunk-aware, motion-aware, and continuation-aware.

B.2 Motivation of Implicit Chunk Scorer

Our implicit chunk scorer is motivated by the theory of implicit process rewards [54]. We briefly review the key idea, because it explains why a log-ratio form can expose dense local signals from only outcome-level supervision.

Implicit process rewards in language models. In language-model alignment, an outcome reward model (ORM) scores a complete response $y = (y_1, \ldots, y_T)$, while a process reward model (PRM) scores intermediate reasoning steps. Explicitly training a PRM usually requires step-level labels, which are expensive to collect. Yuan et al. [54] show that this extra supervision can be avoided if the outcome reward is parameterized as a policy-reference log-likelihood ratio:

$$r_\phi(y) = \beta \log \frac{\pi_\phi(y)}{\pi_{\mathrm{ref}}(y)}. \tag{13}$$

Because an autoregressive likelihood factorizes as

$$\log \frac{\pi_\phi(y)}{\pi_{\mathrm{ref}}(y)} = \sum_{t=1}^{T} \log \frac{\pi_\phi(y_t \mid y_{<t})}{\pi_{\mathrm{ref}}(y_t \mid y_{<t})}, \tag{14}$$

the reward of a partial response can be written as the cumulative prefix log-ratio

$$q_\phi^t(y_{\le t}) = \sum_{\ell=1}^{t} \beta \log \frac{\pi_\phi(y_\ell \mid y_{<\ell})}{\pi_{\text{ref}}(y_\ell \mid y_{<\ell})}. \tag{15}$$

Their main theoretical observation is that this quantity is not merely a heuristic prefix score: under the log-ratio reward parameterization, it corresponds to an entropy-regularized expectation of the final outcome reward under the reference continuation distribution.

$$q_\phi^t(y_{\le t}) = \beta \log \mathbb{E}_{\pi_{\text{ref}}(y_{>t} \mid y_{\le t})} \left[\exp\left(\frac{1}{\beta} r_\phi(y) \right) \right]. \tag{16}$$

Thus q_ϕ^t can be interpreted as a process-level value induced by the outcome reward. The incremental process reward is then:

$$r_\phi^t = q_\phi^t - q_\phi^{t-1} = \beta \log \frac{\pi_\phi(y_t \mid y_{<t})}{\pi_{\text{ref}}(y_t \mid y_{<t})}. \tag{17}$$

This explains why the local reward takes a log-likelihood-ratio form: it is the additive increment of a prefix value whose terminal sum recovers the outcome-level log-ratio reward.

Why the loss can be flexible. The above argument depends primarily on the reward parameterization, not on a particular training objective. In practice, the ORM can be trained with preference objectives such as DPO, or with classification-style losses such as CE, as long as the scalar reward used by the loss is represented by the same log-ratio form. The objective trains the model so that complete responses are ordered correctly, while the autoregressive factorization makes the learned outcome score decomposable into prefix or step increments. This is why implicit PRMs can be obtained from response-level data without explicit process labels. The same work also shows that the induced prefix value lies between soft and hard look-ahead estimates, providing a robustness interpretation relative to MCTS-style process annotation.

Gap to embodied flow-based policies. This theory cannot be directly transferred to embodied flow-based manipulation policies. First, language models provide explicit autoregressive token likelihoods, while a flow policy generates an action chunk through a multi-step solver path; the marginal likelihood of the terminal action generally requires integrating over intermediate solver states and is not tractable. Second, the hierarchy is different: language has response-level outcomes and token/step-level likelihood increments, whereas embodied control has trajectory-level outcomes, action chunks, and inner denoising states. A solver state $x_{i,j}$ is not itself an executed action and does not carry the same semantic status as a reasoning step. Third, the data distribution changes online: the scorer reference b_{m-1} advances with behavior snapshots, while b_m generates the current rollouts and anchors actor mirroring. Finally, manipulation rollouts contain many easy failures and sparse near-success failures, so naively training a scorer from all success–failure pairs may collapse into a coarse trajectory classifier instead of learning local action quality.

Our adaptation. The implicit chunk scorer in Sec. 4.1 keeps the central principle of implicit process rewards—derive dense local preference signals from outcome-level supervision through a log-ratio parameterization—but replaces autoregressive token likelihoods with tractable solver-path density ratios. Specifically, the response likelihood ratio in language models is replaced by a chunk-path ratio over stochastic solver transitions, and the token-level increment is replaced by a chunk-level score accumulated over the denoising path that produces one executed action. The Gaussian local transition identity and the path-lifting derivation in Appendices D.2 and D.3 provide the corresponding flow-policy analogue. To make this scorer reliable in embodied online RL, we further use the hard-failure-centric training protocol in Appendix C, which filters, reweights, and replenishes informative near-miss negatives so that the scorer remains focused on local action-quality differences rather than coarse trajectory separability.

B.3 Chunk Score Normalization and Outcome-Consistency Gate

The raw chunk score $u_{\phi,i}$ can have different scales across tasks and training iterations. We robustly normalize it within each training batch:

$$y_{\phi,i} = \tanh\left(\frac{u_{\phi,i} - \mathrm{median}(\mathcal{U}_\phi)}{\tau_\phi(\mathrm{MAD}(\mathcal{U}_\phi) + \epsilon)}\right),$$

where $\mathcal{U}_\phi = \{u_{\phi,i}\}$ is the set of chunk scores in the current batch. We then define the outcome-consistency gate

$$g_i = \eta_{\min} + (\eta_{\max} - \eta_{\min})\frac{1 + y_\tau y_{\phi,i}}{2}.$$

This gate does not force the chunk label to match the trajectory outcome; it only controls how strongly the chunk signal is trusted in the actor update.

B.4 Pair-Guidance Confidence Weight

Let s_i^g and s_i^b be the scorer values of the selected good and bad suffixes. We use a confidence weight

$$w_{\mathrm{pair},i} = \mathrm{stopgrad}\left[\max\left(0, \frac{s_i^g - s_i^b - \delta_{\mathrm{pair}}}{1 - \delta_{\mathrm{pair}}}\right)\right].$$

Pairs with small scorer gaps are discarded or assigned near-zero weight, which prevents noisy candidate rankings from dominating actor optimization. The stop-gradient operator ensures that this weight is treated as a confidence coefficient rather than another optimization target.

C Hard-Failure-Centric Training of the Implicit Chunk Scorer

C.1 Methodology Explanations

The implicit chunk scorer in Sec. 4.1 is learned from trajectory-level preferences. In embodied manipulation, however, a naive use of all success–failure pairs is often inefficient: many failures are easy negatives, e.g., early deviations, wrong motion directions, or obvious grasp-timing errors. Such pairs are useful for distinguishing successful and failed rollouts, but they provide weak supervision for learning fine-grained chunk quality. In practice, overusing them may make the scorer behave like a coarse trajectory classifier rather than a local action-quality estimator. We therefore adopt a hard-failure-centric training protocol that focuses the scorer on failures that are close to success at the trajectory level but expose local motion defects.

Natural hard failures. At outer iteration m, rollouts are grouped by the same task condition and initial state. For each successful trajectory τ^+, we first select candidate failed trajectories τ^- from the same group. A useful negative should be semantically close to τ^+ while still exhibiting worse local action quality. We therefore define two detached hardness signals. The first one measures semantic hardness:

$$h_{\mathrm{sem}}(\tau^+, \tau^-) = \sigma\left(\frac{\mathrm{Sim}_{\mathrm{traj}}(\tau^+, \tau^-) - \delta_{\mathrm{sem}}}{T_{\mathrm{sem}}}\right) \cdot \mathbb{1}[t_{\mathrm{fail}}(\tau^-) > \alpha H], \tag{18}$$

where $\mathrm{Sim}_{\mathrm{traj}}$ is computed from stage-aligned state or chunk embeddings, t_{fail} denotes the first clear failure time, and the indicator suppresses failures that deviate too early. This favors failures that remain close to successful rollouts for a substantial portion of the task.

The second signal measures quality hardness. It asks whether the failed rollout is locally worse in the motion properties that matter for manipulation, rather than merely different at the trajectory level. We compute

$$h_{\mathrm{qual}}(\tau^+, \tau^-) = \sigma\left(\frac{C_{\mathrm{qual}}(\tau^-, \tau^+) - \delta_{\mathrm{qual}}}{T_{\mathrm{qual}}}\right), \quad C_{\mathrm{qual}} = \sum_\ell \omega_\ell \Delta_\ell(\tau^-, \tau^+), \tag{19}$$

where Δ_ℓ denotes normalized local defect measures, such as path inefficiency, reversal, gripper oscillation, chunk discontinuity, or state-path inconsistency. We apply this quality test after the semantic gate, so that low-level defect scores are not evaluated on obviously irrelevant failures. The combined hard-negative prior is

$$h_{\text{prior}}(\tau^+, \tau^-) = h_{\text{sem}}(\tau^+, \tau^-) h_{\text{qual}}(\tau^+, \tau^-). \tag{20}$$

Intuitively, h_{sem} ensures that the negative is near-successful enough to be informative, while h_{qual} ensures that it actually exposes local motion defects.

Hardness-aware pair weighting. Even after hard-pair filtering, not all pairs should contribute equally. Some pairs become easy as the scorer improves, and repeatedly training on them can waste gradient budget or enlarge already-saturated preference margins. We therefore combine the static hard-negative prior with a dynamic margin-based weight. For a preference pair $p = (\tau^+, \tau^-)$, let

$$M_\phi(p) = \sum_i u^+_{\phi,i} - \sum_i u^-_{\phi,i} \tag{21}$$

be the current scorer margin. Within each rollout group, we percentile-normalize h_{prior} to obtain $\bar{h}_{\text{prior}} \in [0, 1]$, and define

$$w_{\text{pair}}(p) = \text{stopgrad}\left[w_{\min} + (w_{\max} - w_{\min}) \bar{h}_{\text{prior}}(p) \exp(-\gamma [M_\phi(p) - \delta_m]_+)\right]. \tag{22}$$

This weighting preserves high weight for genuine hard negatives, assigns moderate weight to uncertain pairs, and downweights pairs whose preference margin is already large. The scorer objective becomes

$$\mathcal{L}^{\text{hard}}_{\text{sc}}(\phi) = -\mathbb{E}_{p=(\tau^+,\tau^-)} \left[w_{\text{pair}}(p) \log \sigma \left(\sum_i u^+_{\phi,i} - \sum_i u^-_{\phi,i} \right) \right]. \tag{23}$$

Compared with uniformly weighting all success–failure pairs, Eq. (23) keeps the scorer focused on the pairs that still carry useful chunk-level information.

Synthetic near-miss failures and recipe bank. As online training progresses, the policy succeeds more often and natural hard failures become increasingly rare. To maintain a steady supply of informative negatives, we maintain an asynchronous recipe bank $\mathcal{B}$ of near-miss branch perturbations mined from previously observed hard failures. A recipe stores a task stage, a local branching position, a semantic motion block, a perturbation direction, and a perturbation scale:

$$r = (\text{task}, \text{stage}, i, j, h, d, \lambda). \tag{24}$$

Given a successful rollout and a matched local prefix $x_{i,j}$, a recipe generates a candidate near-miss suffix by perturbing the local branch direction in the corresponding semantic subspace:

$$\tilde{v}_{i,j} = v_{b_m,i,j} + \lambda P_h d, \qquad h \in \{\text{tr}, \text{rot}, \text{gr}\}. \tag{25}$$

The perturbed suffix is used as a synthetic hard negative only if it remains semantically close to the original success while receiving a lower local quality score. We do not treat arbitrary synthetic branches as ground-truth failures; they are used as auxiliary near-miss pairs and are periodically refreshed or discarded when their hardness score becomes small.

The recipe bank plays two roles. Early in training, the scorer mainly learns from natural hard failures collected from real rollouts. Later, when genuine failures become scarce, the bank reuses previously validated near-miss patterns to prevent the scorer from losing hard-negative supervision. This provides a low-cost hard-negative supply mechanism without requiring exhaustive additional branch rollouts at every environment step.

Summary. The overall scorer training procedure can be viewed as a four-stage curriculum: first, build trajectory preferences from rollout outcomes; second, retain failures that are semantically close to success but locally worse; third, reweight pairs according to both static hardness and current scorer margin; and finally, use a lightweight recipe bank to replenish near-miss negatives when natural failures become rare. This protocol is not intended to redefine trajectory preference learning. Instead, it adapts implicit chunk scoring to the data distribution of embodied manipulation, where the most useful supervision often comes from near-successful failures that reveal local action-quality differences.

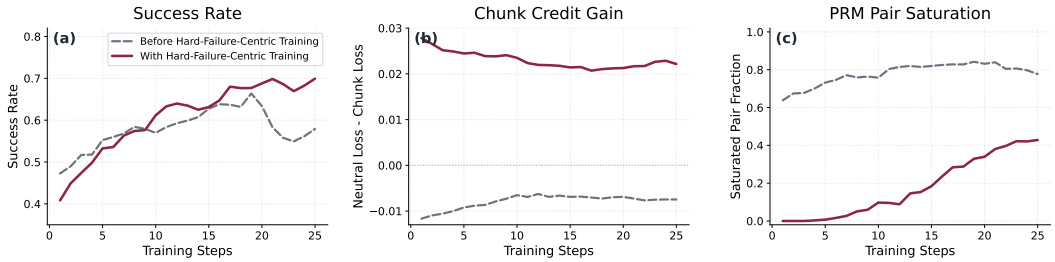


Figure 4: **Hard-failure-centric training is necessary for learning an implicit chunk scorer.** We compare object-task training dynamics before and after introducing our hard-failure-centric training recipe, using the first 25 common training steps for a fair comparison. (a) The policy trained with our recipe improves success rate more rapidly. (b) *Chunk Credit Gain*, defined as $L_{\text{neutral}} - L_{\text{chunk}}$, is negative without the recipe but becomes consistently positive with it, showing that chunk-level supervision only becomes more informative than a neutral baseline after hard-failure-centric scorer training. (c) *PRM Pair Saturation* is substantially reduced, indicating that the scorer no longer collapses into an easy-pair regime early in training. Together, these trends show that our training recipe is essential for keeping the scorer learnable and for converting implicit chunk-quality modeling into effective policy improvement.

C.2 Experiments Analysis

Why hard-failure-centric scorer training is necessary? Figure 4 compares object-task training before and after introducing our hard-failure-centric training recipe for the implicit chunk scorer. Without this recipe, the scorer quickly collapses into an easy-pair regime: *PRM Pair Saturation* rises to a high level almost immediately, while *Chunk Credit Gain* remains unfavorable relative to a neutral chunk baseline. As a result, chunk-level supervision does not become a reliable learning signal, and policy improvement is slower. In contrast, our training recipe keeps pair saturation substantially lower in the same early training window, while turning chunk credit gain consistently positive. This indicates that hard negative selection, weighted pair construction, and controlled synthetic failures jointly help keep the scorer in a learnable regime, so that chunk-level quality signals can be effectively transferred to the policy and improve both success rate and adaptation speed.

Additional scorer diagnostics. Figure 5 provides a complementary view of the same phenomenon. Before hard-failure-centric training, the scorer produces excessively large *PRM Pair Margin* values in the first few updates, together with persistently high pair saturation, indicating that optimization is dominated by trivial success–failure separations rather than informative near-miss distinctions. After introducing our training recipe, pair margins remain much more controlled in the same early-stage window, and saturation is correspondingly reduced. This behavior is consistent with the intended role of our method: it shifts scorer learning away from abundant easy failures and toward hard, locally informative comparisons, which is precisely the condition required for implicit chunk-quality modeling to become useful for embodied NFT adaptation.

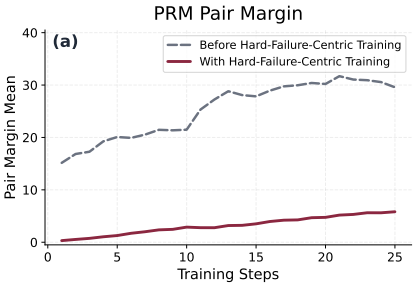


Figure 5: **Controlled pair margins under hard-failure-centric training.** Without hard-failure-centric training, pair margins grow excessively large almost immediately. With our recipe, pair margins remain substantially more controlled.

C.3 Task-Grounded Validation of Chunk Scores

We freeze the scorer and evaluate held-out same-prefix perturbation pairs excluded from training. Score gaps correlate with simulator short-horizon predicate progress and identify the continuation

with higher downstream progress/success (Table 6). The AUC confidence interval is obtained by trajectory bootstrap.

Table 6: Task-grounded validation of the frozen chunk scorer on held-out same-prefix perturbation pairs.

Validation target	Metric	Result
Short-horizon predicate progress	Spearman ρ	0.83
Higher downstream progress/success	AUC	0.86 (CI [0.82, 0.91])

D Theoretical Foundations and Proofs

This appendix provides the technical derivations used in Secs. 3.1 and 4.1. We first derive the affine form of the local Gaussian mean induced by the stochastic solver step, then prove the local log-density-ratio identity, and finally show how solver-level density ratios lift to chunk-level and rollout-level additive scores.

D.1 Affine Mean Induced by the Euler–Maruyama Solver

Recall the stochastic solver update in Eq. (1). For a generic velocity field v_ω, the Euler–Maruyama transition can be written as:

$$x^{\rho^-} = x^\rho + \left[v_\omega(x^\rho, \rho, c) + \frac{\sigma_\rho^2}{2\rho}\big(x^\rho + (1-\rho)v_\omega(x^\rho, \rho, c)\big) \right](-\Delta\rho) + \sigma_\rho\sqrt{\Delta\rho}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \tag{26}$$

Since the noise is Gaussian, this defines a local Gaussian transition:

$$p_{\omega,\rho}(x^{\rho^-} \mid x^\rho, c) = \mathcal{N}(\mu_{\omega,\rho}(x^\rho, c), \Sigma_\rho), \qquad \Sigma_\rho = \sigma_\rho^2 \Delta\rho I \tag{27}$$

Expanding the deterministic part gives:

$$x^{\rho^-} = \left(1 - \frac{\sigma_\rho^2}{2\rho}\Delta\rho\right) x^\rho - \Delta\rho\left(1 + \frac{\sigma_\rho^2}{2\rho}(1-\rho)\right) v_\omega(x^\rho, \rho, c) + \sigma_\rho\sqrt{\Delta\rho}\epsilon. \tag{28}$$

Taking conditional expectation over ϵ yields:

$$\mu_{\omega,\rho}(x^\rho, c) = \left(1 - \frac{\sigma_\rho^2}{2\rho}\Delta\rho\right) x^\rho - \Delta\rho\left(1 + \frac{\sigma_\rho^2}{2\rho}(1-\rho)\right) v_\omega(x^\rho, \rho, c). \tag{29}$$

Thus the local mean is affine in the velocity field:

$$\mu_{\omega,\rho}(x^\rho, c) = M_\rho v_\omega(x^\rho, \rho, c) + W_\rho(x^\rho, \rho),$$

where

$$M_\rho = -\Delta\rho\left(1 + \frac{\sigma_\rho^2}{2\rho}(1-\rho)\right), \qquad W_\rho(x^\rho, \rho) = \left(1 - \frac{\sigma_\rho^2}{2\rho}\Delta\rho\right) x^\rho.$$

This proves Eq. (2). The affine form is used in later proofs to express local density ratios and mirrored margins as quadratic energy differences.

D.2 Proof of the Local Log-Density-Ratio Identity

Fix a chunk i, solver step j, and observed local transition $x_{i,j} \to x_{i,j+1}$. Let two candidate models ω and $\bar{\omega}$ induce Gaussian local transitions with the same covariance:

$$p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i) = \mathcal{N}(\mu_{\omega,\rho_j}, \Sigma_{\rho_j}),$$

$$p_{\bar{\omega},\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i) = \mathcal{N}(\mu_{\bar{\omega},\rho_j}, \Sigma_{\rho_j}).$$

For any target z, recall the local transition energy

$$e_{\omega,i,j}(z) := \|z - \mu_{\omega,\rho_j}(x_{i,j}, c_i)\|^2_{\Sigma^{-1}_{\rho_j}}.$$

Applying the Gaussian log-density formula to the observed target $z = x_{i,j+1}$, we have:

$$\log p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\log\det\Sigma_{\rho_j} - \frac{1}{2}e_{\omega,i,j}(x_{i,j+1}), \tag{30}$$

and analogously for $\bar{\omega}$. Subtracting the two log-densities cancels the normalization constants because the covariance is shared:

$$\log\frac{p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)}{p_{\bar{\omega},\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)} = -\frac{1}{2}\left[e_{\omega,i,j}(x_{i,j+1}) - e_{\bar{\omega},i,j}(x_{i,j+1})\right]. \tag{31}$$

This proves Eq. (4). The identity holds for any pair of models or branches sharing the same local covariance.

D.3 From Solver-Step Densities to Chunk and Trajectory Scores

We prove that the solver-step density-ratio identity in Eq. (4) induces an additive chunk-level score and, after summing over chunks, a trajectory-level preference score. The proof proceeds by explicitly tracking the products of local transition densities and the corresponding log-ratio sums.

Step 1: path density inside one action chunk. For the i-th action chunk, the stochastic solver generates a latent path

$$\zeta_i = (x_{i,0}, x_{i,1}, \ldots, x_{i,K}), \qquad a_i = x_{i,K}.$$

Conditioned on the initial solver state $x_{i,0}$ and the control context c_i, the solver dynamics are Markov along the inner solver index j. Therefore, by repeated application of the chain rule,

$$\begin{aligned} &p_\omega(x_{i,1}, x_{i,2}, \ldots, x_{i,K} \mid x_{i,0}, c_i) \\ &\quad = p_{\omega,\rho_0}(x_{i,1} \mid x_{i,0}, c_i)p_{\omega,\rho_1}(x_{i,2} \mid x_{i,1}, c_i)\cdots p_{\omega,\rho_{K-1}}(x_{i,K} \mid x_{i,K-1}, c_i) \\ &\quad = \prod_{j=0}^{K-1} p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i). \end{aligned} \tag{32}$$

Throughout, initial solver states are conditioned on and omitted from the notation. We denote this conditional solver-path density by

$$\mathcal{Q}_\omega(\zeta_i \mid c_i) := \prod_{j=0}^{K-1} p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i). \tag{33}$$

This is a path likelihood over the realized denoising trajectory, not the marginal density of the terminal action. Indeed, the marginal action density would require integrating out all intermediate solver states:

$$p_\omega(a_i = x_{i,K} \mid x_{i,0}, c_i) = \int \prod_{j=0}^{K-1} p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)\, dx_{i,1}\cdots dx_{i,K-1}, \tag{34}$$

which is generally intractable for a learned nonlinear flow policy. Our construction therefore works with the tractable pathwise likelihood ratio.

Step 2: from local ratios to the chunk-path ratio. Consider two solver models ω and $\bar{\omega}$ evaluated on the same realized solver path ζ_i. Using Eq. (33), their path-likelihood ratio is

$$\frac{\mathcal{Q}_\omega(\zeta_i \mid c_i)}{\mathcal{Q}_{\bar{\omega}}(\zeta_i \mid c_i)} = \frac{\prod_{j=0}^{K-1} p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)}{\prod_{j=0}^{K-1} p_{\bar{\omega},\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)} = \prod_{j=0}^{K-1} \frac{p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)}{p_{\bar{\omega},\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)}. \tag{35}$$

Taking the logarithm converts the product of local ratios into an additive sum:

$$\log \frac{\mathcal{Q}_\omega(\zeta_i \mid c_i)}{\mathcal{Q}_{\bar{\omega}}(\zeta_i \mid c_i)} = \log \prod_{j=0}^{K-1} \frac{p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)}{p_{\bar{\omega},\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)} = \sum_{j=0}^{K-1} \log \frac{p_{\omega,\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)}{p_{\bar{\omega},\rho_j}(x_{i,j+1} \mid x_{i,j}, c_i)}. \tag{36}$$

Now applying the local density-ratio identity in Eq. (4) to every solver transition yields

$$\begin{aligned} \log \frac{\mathcal{Q}_\omega(\zeta_i \mid c_i)}{\mathcal{Q}_{\bar{\omega}}(\zeta_i \mid c_i)} &= \sum_{j=0}^{K-1} \left[-\frac{1}{2} \left(e_{\omega,i,j}(x_{i,j+1}) - e_{\bar{\omega},i,j}(x_{i,j+1}) \right) \right] \\ &= -\frac{1}{2} \sum_{j=0}^{K-1} \left[e_{\omega,i,j}(x_{i,j+1}) - e_{\bar{\omega},i,j}(x_{i,j+1}) \right]. \end{aligned} \tag{37}$$

Thus, the chunk-path log-ratio is exactly the accumulated solver-step energy gap.

Step 3: from chunk ratios to rollout ratios. A full rollout contains both solver-path generation and environment transitions. Conditioned on the initial solver states, with each c_i determined by the observed history and task, the joint density of the rollout and all solver paths factorizes as

$$p_\omega(\tau, \zeta_{0:H-1} \mid s_0, c) = \prod_{i=0}^{H-1} \mathcal{Q}_\omega(\zeta_i \mid c_i) p(s_{i+1} \mid s_i, a_i), \qquad a_i = x_{i,K}. \tag{38}$$

For the same realized rollout, comparing ω and $\bar{\omega}$ gives

$$\begin{aligned} \frac{p_\omega(\tau, \zeta_{0:H-1} \mid s_0, c)}{p_{\bar{\omega}}(\tau, \zeta_{0:H-1} \mid s_0, c)} &= \frac{\prod_{i=0}^{H-1} \mathcal{Q}_\omega(\zeta_i \mid c_i) p(s_{i+1} \mid s_i, a_i)}{\prod_{i=0}^{H-1} \mathcal{Q}_{\bar{\omega}}(\zeta_i \mid c_i) p(s_{i+1} \mid s_i, a_i)} \\ &= \prod_{i=0}^{H-1} \frac{\mathcal{Q}_\omega(\zeta_i \mid c_i)}{\mathcal{Q}_{\bar{\omega}}(\zeta_i \mid c_i)} \cdot \prod_{i=0}^{H-1} \frac{p(s_{i+1} \mid s_i, a_i)}{p(s_{i+1} \mid s_i, a_i)}. \end{aligned} \tag{39}$$

The second product is exactly one because the environment transition kernel does not depend on the compared solver model once the same state-action trajectory is fixed. Hence,

$$\frac{p_\omega(\tau, \zeta_{0:H-1} \mid s_0, c)}{p_{\bar{\omega}}(\tau, \zeta_{0:H-1} \mid s_0, c)} = \prod_{i=0}^{H-1} \frac{\mathcal{Q}_\omega(\zeta_i \mid c_i)}{\mathcal{Q}_{\bar{\omega}}(\zeta_i \mid c_i)}. \tag{40}$$

Taking logarithms again converts the product over chunks into a sum:

$$\log \frac{p_\omega(\tau, \zeta_{0:H-1} \mid s_0, c)}{p_{\bar{\omega}}(\tau, \zeta_{0:H-1} \mid s_0, c)} = \log \prod_{i=0}^{H-1} \frac{\mathcal{Q}_\omega(\zeta_i \mid c_i)}{\mathcal{Q}_{\bar{\omega}}(\zeta_i \mid c_i)} = \sum_{i=0}^{H-1} \log \frac{\mathcal{Q}_\omega(\zeta_i \mid c_i)}{\mathcal{Q}_{\bar{\omega}}(\zeta_i \mid c_i)}. \tag{41}$$

Combining Eq. (41) with Eq. (37), we obtain the full solver-path energy representation:

$$\log \frac{p_\omega(\tau, \zeta_{0:H-1} \mid s_0, c)}{p_{\bar{\omega}}(\tau, \zeta_{0:H-1} \mid s_0, c)} = -\frac{1}{2} \sum_{i=0}^{H-1} \sum_{j=0}^{K-1} \left[e_{\omega,i,j}(x_{i,j+1}) - e_{\bar{\omega},i,j}(x_{i,j+1}) \right]. \tag{42}$$

This equation is the desired lifting result: a rollout-level model comparison decomposes into a double sum of solver-step energy gaps.

Step 4: implicit chunk score as the additive unit. We now instantiate the two compared models as the trainable scorer $\phi^{(m)}$ and the previous rollout snapshot b_{m-1}. From Eq. (37), the chunk-level contribution is

$$\begin{aligned} u_{\phi,i}^{(m)} &:= \beta \log \frac{\mathcal{Q}_{\phi^{(m)}}(\zeta_i \mid c_i)}{\mathcal{Q}_{b_{m-1}}(\zeta_i \mid c_i)} \\ &= -\frac{\beta}{2} \sum_{j=0}^{K-1} \left[e_{\phi^{(m)},i,j}(x_{i,j+1}) - e_{b_{m-1},i,j}(x_{i,j+1}) \right], \end{aligned} \tag{43}$$

which is Eq. (6). Summing this quantity over the rollout gives

$$\begin{aligned}\sum_{i=0}^{H-1} u_{\phi,i}^{(m)} &= \beta \sum_{i=0}^{H-1} \log \frac{\mathcal{Q}_{\phi^{(m)}}(\zeta_i \mid c_i)}{\mathcal{Q}_{b_{m-1}}(\zeta_i \mid c_i)} \\ &= \beta \log \prod_{i=0}^{H-1} \frac{\mathcal{Q}_{\phi^{(m)}}(\zeta_i \mid c_i)}{\mathcal{Q}_{b_{m-1}}(\zeta_i \mid c_i)} = \beta \log \frac{p_{\phi^{(m)}}(\tau, \zeta_{0:H-1} \mid s_0, c)}{p_{b_{m-1}}(\tau, \zeta_{0:H-1} \mid s_0, c)}.\end{aligned} \tag{44}$$

The last equality uses the environment-transition cancellation in Eq. (40). Therefore, $u_{\phi,i}^{(m)}$ is the additive chunk-level unit whose sum recovers the rollout-level solver-path log-ratio.

Step 5: Bradley–Terry preference argument. For a preference pair (τ^+, τ^-), define the rollout score

$$R_\phi(\tau) := \sum_{i=0}^{H-1} u_{\phi,i}^{(m)}. \tag{45}$$

Under a Bradley–Terry preference model, the probability that τ^+ is preferred over τ^- is

$$P_\phi(\tau^+ \succ \tau^-) = \sigma\big(R_\phi(\tau^+) - R_\phi(\tau^-)\big)\,. \tag{46}$$

Substituting Eq. (45) gives

$$P_\phi(\tau^+ \succ \tau^-) = \sigma\left(\sum_{i=0}^{H-1} u_{\phi,i}^{+} - \sum_{i=0}^{H-1} u_{\phi,i}^{-}\right). \tag{47}$$

The negative log-likelihood of this preference model is precisely

$$\mathcal{L}_{\mathrm{sc}}(\phi) = -\mathbb{E}_{(\tau^+,\tau^-)}\left[\log \sigma\left(\sum_i u_{\phi,i}^{+} - \sum_i u_{\phi,i}^{-}\right)\right], \tag{48}$$

which matches Eq. (7). Hence, the scorer objective is not an arbitrary auxiliary classifier: it is the Bradley–Terry likelihood induced by a rollout score that decomposes additively into chunk-level solver-path density ratios.

This completes the derivation from local solver-step densities to chunk-level implicit scores and finally to trajectory-level preference learning.

D.4 Proof of the Exact Semantic Decomposition

We prove the decomposition used in Eq. (9). The key observation is that the symmetric positive-vs-negative mirrored margin cancels all quadratic terms and leaves a linear functional of the local update direction. Therefore, decomposing the update direction by coordinate projectors induces an additive decomposition of the margin. If each block is evaluated with a separate projected covariance, the same equality holds under a block-compatible metric; otherwise the block construction should be viewed as the stable structured surrogate analyzed in Appendix D.5.

Setup. Fix chunk i and solver step j. For compactness, write

$$v_b := v_{b_m}(x_{i,j}, \rho_j, c_i), \qquad \Delta v := v_\theta(x_{i,j}, \rho_j, c_i) - v_b. \tag{49}$$

By the affine velocity-to-mean map in Eq. (2), the Gaussian mean induced by a velocity field v can be written as

$$\mu_{i,j}(v) = M_{\rho_j} v + W_{\rho_j}(x_{i,j}, \rho_j), \tag{50}$$

where M_{ρ_j} is the solver-dependent velocity coefficient and W_{ρ_j} does not depend on v. Define the residual under the behavior snapshot and the local covariance as

$$r_{i,j} := x_{i,j+1} - \mu_{i,j}(v_b), \qquad \Sigma_j := \Sigma_{\rho_j}. \tag{51}$$

Step 1: expanding the full mirrored margin. The full and subspace probes use the same amplitude η_j. The full mirrored branches are

$$v_{i,j}^{+} = v_b + \eta_j \Delta v, \qquad v_{i,j}^{-} = v_b - \eta_j \Delta v. \tag{52}$$

Using the affine mean map,

$$\mu_{i,j}(v_{i,j}^{+}) = M_{\rho_j}(v_b + \eta_j \Delta v) + W_{\rho_j} = \mu_{i,j}(v_b) + M_{\rho_j}\eta_j \Delta v, \tag{53}$$

and similarly

$$\mu_{i,j}(v_{i,j}^{-}) = \mu_{i,j}(v_b) - M_{\rho_j}\eta_j \Delta v. \tag{54}$$

Let

$$d_{i,j} := M_{\rho_j}\eta_j \Delta v. \tag{55}$$

Then the two residuals against the observed target $x_{i,j+1}$ are

$$x_{i,j+1} - \mu_{i,j}(v_{i,j}^{+}) = r_{i,j} - d_{i,j}, \qquad x_{i,j+1} - \mu_{i,j}(v_{i,j}^{-}) = r_{i,j} + d_{i,j}. \tag{56}$$

Hence the full mirrored gap is

$$\delta_{i,j}^{\text{full}} = \|r_{i,j} - d_{i,j}\|_{\Sigma_j^{-1}}^2 - \|r_{i,j} + d_{i,j}\|_{\Sigma_j^{-1}}^2. \tag{57}$$

We now expand both quadratic forms:

$$\begin{aligned} \|r_{i,j} - d_{i,j}\|_{\Sigma_j^{-1}}^2 &= (r_{i,j} - d_{i,j})^\top \Sigma_j^{-1}(r_{i,j} - d_{i,j}) \\ &= r_{i,j}^\top \Sigma_j^{-1} r_{i,j} - r_{i,j}^\top \Sigma_j^{-1} d_{i,j} - d_{i,j}^\top \Sigma_j^{-1} r_{i,j} + d_{i,j}^\top \Sigma_j^{-1} d_{i,j}. \end{aligned} \tag{58}$$

Since Σ_j^{-1} is symmetric, $r_{i,j}^\top \Sigma_j^{-1} d_{i,j} = d_{i,j}^\top \Sigma_j^{-1} r_{i,j}$, so

$$\|r_{i,j} - d_{i,j}\|_{\Sigma_j^{-1}}^2 = r_{i,j}^\top \Sigma_j^{-1} r_{i,j} - 2 r_{i,j}^\top \Sigma_j^{-1} d_{i,j} + d_{i,j}^\top \Sigma_j^{-1} d_{i,j}. \tag{59}$$

Similarly,

$$\|r_{i,j} + d_{i,j}\|_{\Sigma_j^{-1}}^2 = r_{i,j}^\top \Sigma_j^{-1} r_{i,j} + 2 r_{i,j}^\top \Sigma_j^{-1} d_{i,j} + d_{i,j}^\top \Sigma_j^{-1} d_{i,j}. \tag{60}$$

Substituting the two expansions into Eq. (57), the residual norm and the quadratic update norm cancel:

$$\begin{aligned} \delta_{i,j}^{\text{full}} &= \left[r_{i,j}^\top \Sigma_j^{-1} r_{i,j} - 2 r_{i,j}^\top \Sigma_j^{-1} d_{i,j} + d_{i,j}^\top \Sigma_j^{-1} d_{i,j}\right] - \left[r_{i,j}^\top \Sigma_j^{-1} r_{i,j} + 2 r_{i,j}^\top \Sigma_j^{-1} d_{i,j} + d_{i,j}^\top \Sigma_j^{-1} d_{i,j}\right] \\ &= -4 r_{i,j}^\top \Sigma_j^{-1} d_{i,j}. \end{aligned} \tag{61}$$

Using $d_{i,j} = M_{\rho_j}\eta_j \Delta v$, we obtain

$$\delta_{i,j}^{\text{full}} = -4 M_{\rho_j}\eta_j r_{i,j}^\top \Sigma_j^{-1} \Delta v. \tag{62}$$

Thus the full mirrored margin is a linear functional of the local update direction Δv. This linearity is what makes semantic decomposition possible.

Step 2: block-wise decomposition. Let $\mathcal{H} = \{\text{tr}, \text{rot}, \text{gr}\}$ and let P_h be coordinate projectors satisfying

$$\sum_{h \in \mathcal{H}} P_h = I, \qquad P_h P_{h'} = 0 \quad (h \neq h'). \tag{63}$$

The block-wise mirrored branches perturb only the h-th semantic subspace:

$$v_{i,j}^{h,+} = v_b + \eta_j P_h \Delta v, \qquad v_{i,j}^{h,-} = v_b - \eta_j P_h \Delta v. \tag{64}$$

Repeating the same expansion as in Step 1 with

$$d_{i,j,h} := M_{\rho_j}\eta_j P_h \Delta v \tag{65}$$

gives

$$\begin{aligned} \delta_{i,j,h}^{\text{blk}} &= \|r_{i,j} - d_{i,j,h}\|_{\Sigma_j^{-1}}^2 - \|r_{i,j} + d_{i,j,h}\|_{\Sigma_j^{-1}}^2 \\ &= -4 r_{i,j}^\top \Sigma_j^{-1} d_{i,j,h} \\ &= -4 M_{\rho_j}\eta_j r_{i,j}^\top \Sigma_j^{-1} P_h \Delta v. \end{aligned} \tag{66}$$

Summing Eq. (66) over all semantic blocks yields

$$\begin{aligned}\sum_{h\in\mathcal{H}}\delta^{\mathrm{blk}}_{i,j,h} &= -4M_{\rho_j}\eta_j\sum_{h\in\mathcal{H}} r_{i,j}^{\top}\Sigma_j^{-1}P_h\Delta v\\ &= -4M_{\rho_j}\eta_j r_{i,j}^{\top}\Sigma_j^{-1}\left(\sum_{h\in\mathcal{H}}P_h\right)\Delta v\\ &= -4M_{\rho_j}\eta_j r_{i,j}^{\top}\Sigma_j^{-1}\Delta v\\ &= \delta^{\mathrm{full}}_{i,j},\end{aligned} \tag{67}$$

where the third equality uses $\sum_h P_h = I$ and the last equality uses Eq. (62). Therefore, the full mirrored margin is exactly the sum of block-wise mirrored margins.

Step 3: axis-wise decomposition inside each block. For a block h, let $\mathcal{A}_h$ denote its axis set and let P_d be the corresponding axis projectors. The atomic gripper uses $\mathcal{A}_{\mathrm{gr}} = \{g\}$ and $P_g = P_{\mathrm{gr}}$. We assume

$$P_h = \sum_{d\in\mathcal{A}_h} P_d, \qquad P_d P_{d'} = 0 \quad (d \neq d'). \tag{68}$$

For an axis $d \in \mathcal{A}_h$, define

$$d_{i,j,d} := M_{\rho_j}\eta_j P_d \Delta v. \tag{69}$$

The axis-wise mirrored gap is

$$\begin{aligned}\delta^{\mathrm{ax}}_{i,j,d} &= \|r_{i,j} - d_{i,j,d}\|^2_{\Sigma_j^{-1}} - \|r_{i,j} + d_{i,j,d}\|^2_{\Sigma_j^{-1}}\\ &= -4M_{\rho_j}\eta_j r_{i,j}^{\top}\Sigma_j^{-1}P_d\Delta v.\end{aligned} \tag{70}$$

Summing over all axes inside block h,

$$\begin{aligned}\sum_{d\in\mathcal{A}_h}\delta^{\mathrm{ax}}_{i,j,d} &= -4M_{\rho_j}\eta_j\sum_{d\in\mathcal{A}_h} r_{i,j}^{\top}\Sigma_j^{-1}P_d\Delta v\\ &= -4M_{\rho_j}\eta_j r_{i,j}^{\top}\Sigma_j^{-1}\left(\sum_{d\in\mathcal{A}_h}P_d\right)\Delta v\\ &= -4M_{\rho_j}\eta_j r_{i,j}^{\top}\Sigma_j^{-1}P_h\Delta v\\ &= \delta^{\mathrm{blk}}_{i,j,h}.\end{aligned} \tag{71}$$

Combining Eqs. (67) and (71), we obtain

$$\delta^{\mathrm{full}}_{i,j} = \sum_{h\in\mathcal{H}}\delta^{\mathrm{blk}}_{i,j,h}, \qquad \delta^{\mathrm{blk}}_{i,j,h} = \sum_{d\in\mathcal{A}_h}\delta^{\mathrm{ax}}_{i,j,d}, \tag{72}$$

which proves Eq. (9).

Remark on projected metrics. The derivation above evaluates all block and axis probes with the same full local metric Σ_j^{-1}. If one instead computes each block margin using a separate projected covariance, the exact equality requires the metric to be compatible with the coordinate decomposition, e.g., diagonal or block-diagonal in the canonical action basis. When this condition is not enforced, the semantic construction remains a structured surrogate: it no longer claims exact equality to the full-space margin, but still provides a bounded local preference signal, with sign consistency under aligned evidence as shown in Appendix D.5.

This proof explains why the decomposition is not merely a heuristic partition of the action vector. The symmetric mirrored margin converts local policy improvement into a linear evidence term, and the semantic projectors assign this evidence to physically meaningful motion subspaces.

D.5 Proof of the Stable Semantic Surrogate

We now provide the complete construction of the semantic margin and prove its stability properties. This subsection serves both as the implementation detail for semantic weights and as the proof of the boundedness and sign-consistency claims in Sec. 4.2.

Step 1: weight construction. Let

$$\tilde{y}_i := g_i y_{\phi,i}$$

be the gated local supervision signal. For each block $h \in \mathcal{H}$, define an agreement score:

$$q_{i,h} = \mathrm{softplus}\left(-\frac{\tilde{y}_i\, \bar{\delta}^{\mathrm{blk}}_{i,h}}{\tau_{\mathrm{blk}}}\right), \qquad \omega_{i,h} = \frac{q_{i,h}}{\sum_{h' \in \mathcal{H}} q_{i,h'} + \epsilon}. \tag{73}$$

For each axis $d \in \mathcal{A}_h$, define:

$$q_{i,d|h} = \mathrm{softplus}\left(-\frac{\tilde{y}_i\, \bar{\delta}^{\mathrm{ax}}_{i,d}}{\tau_{\mathrm{ax}}}\right), \qquad \omega_{i,d|h} = \frac{q_{i,d|h}}{\sum_{d' \in \mathcal{A}_h} q_{i,d'|h} + \epsilon}. \tag{74}$$

With $\epsilon > 0$ and $0 \le \lambda_{\mathrm{ax}} \le 1$, the semantic margin is:

$$\bar{\delta}^{\mathrm{sem}}_i = \sum_{h \in \mathcal{H}} \omega_{i,h} \left[(1-\lambda_{\mathrm{ax}})\bar{\delta}^{\mathrm{blk}}_{i,h} + \lambda_{\mathrm{ax}} \sum_{d \in \mathcal{A}_h} \omega_{i,d|h} \bar{\delta}^{\mathrm{ax}}_{i,d}\right]. \tag{75}$$

Step 2: positivity and normalization. Since $\mathrm{softplus}(z) > 0$, all q's are positive. Therefore,

$$\omega_{i,h} \ge 0, \qquad \sum_{h \in \mathcal{H}} \omega_{i,h} \le 1,$$

and for each block,

$$\omega_{i,d|h} \ge 0, \qquad \sum_{d \in \mathcal{A}_h} \omega_{i,d|h} \le 1.$$

Define the effective mixture weights

$$\Omega^{\mathrm{blk}}_{i,h} := \omega_{i,h}(1-\lambda_{\mathrm{ax}}), \qquad \Omega^{\mathrm{ax}}_{i,h,d} := \omega_{i,h}\lambda_{\mathrm{ax}}\omega_{i,d|h}.$$

All effective weights are nonnegative, and

$$\sum_h \Omega^{\mathrm{blk}}_{i,h} + \sum_h \sum_{d \in \mathcal{A}_h} \Omega^{\mathrm{ax}}_{i,h,d} = (1-\lambda_{\mathrm{ax}}) \sum_h \omega_{i,h} + \lambda_{\mathrm{ax}} \sum_h \omega_{i,h} \sum_{d \in \mathcal{A}_h} \omega_{i,d|h} \le 1. \tag{76}$$

The remaining weight can be assigned to zero, making $\bar{\delta}^{\mathrm{sem}}_i$ a convex combination of zero and the block and axis probe margins.

Step 3: boundedness. Thus $\bar{\delta}^{\mathrm{sem}}_i$ lies in the convex hull of zero and its constituent margins. Hence,

$$\left|\bar{\delta}^{\mathrm{sem}}_i\right| \le \max\left\{\max_h |\bar{\delta}^{\mathrm{blk}}_{i,h}|,\ \max_{h,d} |\bar{\delta}^{\mathrm{ax}}_{i,d}|\right\}. \tag{77}$$

Therefore, the semantic margin cannot exceed the largest active probe margin in magnitude, which prevents the semantic reweighting from amplifying unstable local evidence.

Step 4: sign consistency. Assume the active block and axis evidence is aligned with the local supervision signal:

$$\tilde{y}_i \bar{\delta}^{\mathrm{blk}}_{i,h} \le 0, \qquad \tilde{y}_i \bar{\delta}^{\mathrm{ax}}_{i,d} \le 0$$

for all active h, d. Multiplying the semantic margin by $\tilde{y}_i$ gives:

$$\tilde{y}_i \bar{\delta}^{\mathrm{sem}}_i = \sum_h \Omega^{\mathrm{blk}}_{i,h} (\tilde{y}_i \bar{\delta}^{\mathrm{blk}}_{i,h}) + \sum_h \sum_{d \in \mathcal{A}_h} \Omega^{\mathrm{ax}}_{i,h,d} (\tilde{y}_i \bar{\delta}^{\mathrm{ax}}_{i,d}). \tag{78}$$

Each effective weight is nonnegative and each bracketed term is nonpositive by assumption. Hence

$$\tilde{y}_i \bar{\delta}^{\mathrm{sem}}_i \le 0.$$

This proves that the semantic margin preserves the preference sign whenever the active semantic evidence is aligned with the local chunk signal.

Conclusion. The semantic margin is therefore bounded and preserves the preference sign under aligned block and axis evidence. This justifies using it to replace the full-space mirrored gap in the trajectory-level and chunk-level actor losses.

D.6 Proof of Conditional Local Preference Interpretation

We derive the Bradley–Terry pair loss from local mirrored density ratios, then extend it to the semantic support scores used in Eq. (11). The proof clarifies why minimizing the loss assigns stronger positive-vs-negative support to the scorer-preferred suffix under the same solver prefix.

Step 1: local support induced by mirrored branches. Fix chunk i, solver step j, and prefix $x_{i,j}$. The positive and negative mirrored branches induce two Gaussian local kernels:

$$p^{+}_{i,j}(z \mid x_{i,j}, c_i) = \mathcal{N}(\mu^{+}_{i,j}, \Sigma_{\rho_j}), \qquad p^{-}_{i,j}(z \mid x_{i,j}, c_i) = \mathcal{N}(\mu^{-}_{i,j}, \Sigma_{\rho_j}). \tag{79}$$

For any local target z, define the mirrored gap

$$\delta_{i,j}(z) = e_{+,i,j}(z) - e_{-,i,j}(z). \tag{80}$$

By the local density-ratio identity in Eq. (4), we have

$$\log \frac{p^{+}_{i,j}(z \mid x_{i,j}, c_i)}{p^{-}_{i,j}(z \mid x_{i,j}, c_i)} = -\frac{1}{2}\delta_{i,j}(z). \tag{81}$$

We call this quantity the local mirrored support score:

$$m_{i,j}(z) := \log \frac{p^{+}_{i,j}(z \mid x_{i,j}, c_i)}{p^{-}_{i,j}(z \mid x_{i,j}, c_i)} = -\frac{1}{2}\delta_{i,j}(z). \tag{82}$$

Thus, a target has stronger support from the positive branch relative to the negative branch precisely when its mirrored gap is smaller.

Step 2: constructing a same-prefix local preference. Conditional pair guidance samples several suffix chains from the same prefix $x_{i,j}$, under the same state s_i, control context c_i, and solver index j. The learned scorer selects a high-quality suffix and a low-quality suffix. Let their first successor states be

$$z^g := x^g_{i,j+1}, \qquad z^b := x^b_{i,j+1}. \tag{83}$$

The same-prefix construction is important: both z^g and z^b are compared under the same conditioning variables $(x_{i,j}, s_i, c_i, j)$. Therefore, the comparison does not mix different environment states, task contexts, or denoising prefixes; it isolates which local continuation should receive stronger actor support.

The scorer-induced preference is

$$z^g \succ z^b, \tag{84}$$

meaning that the good successor should be more supported by the current actor's positive mirrored branch than the bad successor.

Step 3: Bradley–Terry model over local support scores. We model the probability of the local preference $z^g \succ z^b$ using a Bradley–Terry form over the support scores:

$$P_\theta(z^g \succ z^b \mid x_{i,j}, c_i) = \sigma\big(\kappa \left[m_{i,j}(z^g) - m_{i,j}(z^b)\right]\big), \tag{85}$$

where $\kappa > 0$ controls ranking sharpness. Substituting Eq. (82) gives

$$\begin{aligned} m_{i,j}(z^g) - m_{i,j}(z^b) &= -\frac{1}{2}\delta_{i,j}(z^g) + \frac{1}{2}\delta_{i,j}(z^b) \\ &= -\frac{1}{2}\left[\delta_{i,j}(z^g) - \delta_{i,j}(z^b)\right]. \end{aligned} \tag{86}$$

Hence,

$$P_\theta(z^g \succ z^b \mid x_{i,j}, c_i) = \sigma\Big(-\frac{\kappa}{2}\left[\delta_{i,j}(z^g) - \delta_{i,j}(z^b)\right]\Big). \tag{87}$$

This equation makes the desired ordering explicit:

$$m_{i,j}(z^g) > m_{i,j}(z^b) \quad \Longleftrightarrow \quad \delta_{i,j}(z^g) < \delta_{i,j}(z^b). \tag{88}$$

Therefore, preferring the good suffix is equivalent to making its mirrored gap smaller than that of the bad suffix.

Step 4: negative log-likelihood yields the pair loss. The negative log-likelihood of the preference event $z^g \succ z^b$ is

$$-\log P_\theta(z^g \succ z^b \mid x_{i,j}, c_i) = -\log \sigma\Big(-\frac{\kappa}{2}\left[\delta_{i,j}(z^g) - \delta_{i,j}(z^b)\right]\Big). \tag{89}$$

Using the identity

$$-\log \sigma(-a) = \mathrm{softplus}(a), \tag{90}$$

with

$$a = \frac{\kappa}{2}\left[\delta_{i,j}(z^g) - \delta_{i,j}(z^b)\right], \tag{91}$$

we obtain

$$-\log P_\theta(z^g \succ z^b \mid x_{i,j}, c_i) = \mathrm{softplus}\left(\frac{\kappa}{2}\left[\delta_{i,j}(z^g) - \delta_{i,j}(z^b)\right]\right). \tag{92}$$

Thus, minimizing this objective pushes

$$\delta_{i,j}(z^g) - \delta_{i,j}(z^b) \tag{93}$$

toward negative values, which increases the local support margin

$$m_{i,j}(z^g) - m_{i,j}(z^b). \tag{94}$$

Step 5: semantic pair guidance. In the main method, pair guidance is applied after motion-mode probing. This replaces the full-space gaps by semantic gaps:

$$\delta_{i,j}(z^g) \to \delta^{g,\mathrm{sem}}_{i,j}, \qquad \delta_{i,j}(z^b) \to \delta^{b,\mathrm{sem}}_{i,j}. \tag{95}$$

Define semantic support by $s^{\mathrm{sem}}_{i,j}(z) := -\delta^{\mathrm{sem}}_{i,j}(z)/2$. Applying the same Bradley–Terry model to these scores gives a structured preference surrogate; reweighting does not require equality to the original full-space Gaussian density ratio. The resulting semantic pair loss is

$$L_{\mathrm{pair},i} = w_{\mathrm{pair},i}\,\mathrm{softplus}\left(\frac{\kappa}{2}\left[\delta^{g,\mathrm{sem}}_{i,j} - \delta^{b,\mathrm{sem}}_{i,j}\right]\right), \tag{96}$$

which matches Eq. (11). The confidence weight $w_{\mathrm{pair},i} \geq 0$ does not change the preference direction; it only scales the contribution of confident good/bad pairs.

Conclusion. Conditional pair guidance is a Bradley–Terry preference loss over semantic support scores, motivated by local mirrored density ratios and applied to two scorer-selected continuations sharing the same solver prefix. It strengthens the actor update by explicitly increasing the support margin of the better local continuation over the worse one within the same local denoising neighborhood.

E Experimental Details

E.1 Detailed Experimental Background Setup

Benchmark and Observation Setup. Our experiments follow the same benchmark protocols as prior flow-policy adaptation work, while using the implementation pipeline in RLinf. For LIBERO, [15] our main results are reported on the Spatial, Object, Goal and Long suites, each evaluated over 500 episodes (50 initial states $\times$ 10 subtasks). Each observation contains two RGB

views (a global scene view and a wrist view), a language instruction, and a 7-dimensional proprioceptive state consisting of end-effector position, end-effector orientation, and gripper state. Actions are continuous 7-dimensional end-effector commands. The underlying task success signal is binary; during RL training we use the standard success-based relative reward formulation in RLinf, while all reported evaluation metrics remain success-based. Table 7 lists the evaluation splits. For ManiSkill [16], we adopt the `PutOnPlateInScene` multitask generalization benchmark used in RL4VLA [43], which contains 4,352 compositional tasks formed from 16 objects, 17 receptacles, and 16 tabletop scenes. Observations consist of a single third-person RGB image, a language instruction, and robot proprioceptive/joint state inputs. Actions are continuous robot control commands. The benchmark uses a composite reward designed to discourage degenerate throwing behaviors while preserving the intended long-horizon manipulation structure.

Group	OOD type	Environment / split	Metric aggregation
IND	In-distribution setting	`PutOnPlateInScene25Main-v3-train`	IND
Vision-Language	Unseen instruction phrasing	`PutOnPlateInScene25Instruct-v1-test`	Vision Avg.
	Unseen table background	`PutOnPlateInScene25VisionImage-v1-test`	
	Dynamic foreground texture, weak	`PutOnPlateInScene25VisionTexture03-v1-test`	
	Dynamic foreground texture, strong	`PutOnPlateInScene25VisionTexture05-v1-test`	
	Image-level perturbation, weak	`PutOnPlateInScene25VisionWhole03-v1-test`	
	Image-level perturbation, strong	`PutOnPlateInScene25VisionWhole05-v1-test`	
Semantic	Multi-object confounder	`PutOnPlateInScene25MultiCarrot-v1-test`	Semantic Avg.
	Multi-object confounder	`PutOnPlateInScene25MultiCarrot-v1-train`	
	Distractive receptacle	`PutOnPlateInScene25MultiPlate-v1-test`	
	Distractive receptacle	`PutOnPlateInScene25MultiPlate-v1-train`	
Execution	Mid-episode object reposition	`PutOnPlateInScene25PositionChangeTo-v1-test`	Execution Avg.
	Unseen object initial position	`PutOnPlateInScene25Position-v1-test`	

Table 7: ManiSkill OOD evaluation tasks. We evaluate generalization on visual-language variations, semantic variations, and execution variations under the PutOnPlateInScene25 benchmark.

Generalization Regime and Model Inputs. The ManiSkill benchmark additionally evaluates out-of-distribution generalization along three axes: visual shifts (e.g., unseen tables, texture changes, and image-level perturbations), semantic shifts (e.g., unseen objects, receptacles, and instruction phrasings), and execution shifts (e.g., unseen positions, unseen robot initial poses, and mid-episode object repositioning). For both benchmarks, we use OpenPI π_0 and $\pi_{0.5}$ as the policy backbone, instantiated as flow-based vision-language-action models with a PaliGemma-3B [55] visual-language backbone and a flow-matching action expert. In our implementation, both π_0 and $\pi_{0.5}$ use vision, language, and proprioceptive inputs; this modality choice is kept consistent between SFT initialization and online RL adaptation. The RGB observations are passed through the standard OpenPI image preprocessing pipeline before being consumed by the policy.

Initialization and Online Adaptation. We initialize policies from task-matched OpenPI checkpoints rather than training from scratch. For LIBERO, to avoid saturated initial performance obscuring RL gains, we use pruned few-shot SFT subsets: π_0 is initialized from 58 demonstrations for each of Spatial/Object/Goal and 208 for Long, while $\pi_{0.5}$ uses a unified 40-trajectory few-shot initialization. For ManiSkill, we initialize from the full 16,384-trajectory SFT model because of the substantially broader compositional task space. During online adaptation, we freeze the vision-language backbone and optimize only the action expert, following the standard adaptation pattern used by prior flow-policy baselines. Training is implemented in RLinf [56] with GPU co-location of environment workers, rollout workers, and actor updates to maximize throughput. Unless otherwise noted, the benchmark protocol, evaluation frequency, and rollout budgeting follow prior flow-policy adaptation practice, while the method-specific components of our approach and their hyperparameters are described separately in the appendix and in the hyperparameter table. Each iteration fits the scorer before the actor update, then retains b_m as the next scorer reference before EMA-updating the behavior policy.

E.2 Hyperparameters for Experiments

Table 8 summarizes the training settings. Here, β_{prm} is the scorer scale β in Eq. (6); it is distinct from the NFT branch scale in Sec. 3.2.

Parameters	**LIBERO**								**ManiSkill**	
	π_0				$\pi_{0.5}$				π_0	$\pi_{0.5}$
	Spatial	Object	Goal	Long	Spatial	Object	Goal	Long	Multitask	Multitask
Train epochs	400	400	400	400	400	400	400	400	250	250
Global batch size	1024	1024	1024	1024	1024	1024	1024	1024	2560	2560
Micro batch size	64	64	64	64	64	64	64	64	32	32
Update epochs	2	2	4	4	1	1	3	4	5	5
Actor lr	1e-5	1e-5	1e-5	1e-5	1e-5	1e-5	1e-5	1e-5	8e-6	8e-6
Value lr (SDE-PPO)	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4	1e-4	1.55e-4	1.55e-4
Interaction steps	240	240	320	480	240	240	320	480	60	60
Parallel environments	64	64	64	64	64	64	64	64	128	128
Rollout epochs	8	8	8	8	8	8	8	8	10	10
Chunk length L	5	5	5	10	5	5	5	10	5	5
Denoise steps K	4	4	4	4	4	4	4	4	4	4
Noise level σ	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
Group size G	32	32	32	32	32	32	32	32	32	32
PRM learning rate	1e-5	1e-5	1e-5	1e-5	1e-5	1e-5	1e-5	1e-5	8e-6	8e-6
PRM β_{prm}	5e-4	5e-4	5e-4	5e-4	5e-4	5e-4	5e-4	5e-4	5e-4	5e-4
PRM energy batch size	4	4	4	4	4	4	4	4	4	4
Synthetic pair weight	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Pair threshold δ_{pair}	3e-4	3e-4	3e-4	3e-4	3e-4	3e-4	3e-4	3e-4	3e-4	3e-4
Chunk weight λ_ϕ	$0.2 \to 1.0$				$0.2 \to 1.0$				$0.2 \to 1.0$	$0.2 \to 1.0$
Pair weight λ_{pair}	$0.02 \to 0.2$				$0.02 \to 0.2$				$0.02 \to 0.2$	$0.02 \to 0.2$
Pair warmup steps	$5 \to 20$				$5 \to 20$				$5 \to 20$	$5 \to 20$

Table 8: Training settings on LIBERO and ManiSkill. Value learning rates apply only to SDE-PPO. For the likelihood-based baselines, LIBERO uses chunk-level log-probabilities and token-level entropy; ManiSkill uses token-level log-probabilities and chunk-level entropy. PAWN uses mirrored energy gaps. Shared NFT settings include branch scale $\beta_{\mathrm{NFT}} = 1.0$ and behavior-policy EMA decay from $\alpha_0 = 0.1$ to $\alpha_{-1} = 0.995$.

E.3 Additional Experimental Results

Figure 6 provides additional LIBERO results.

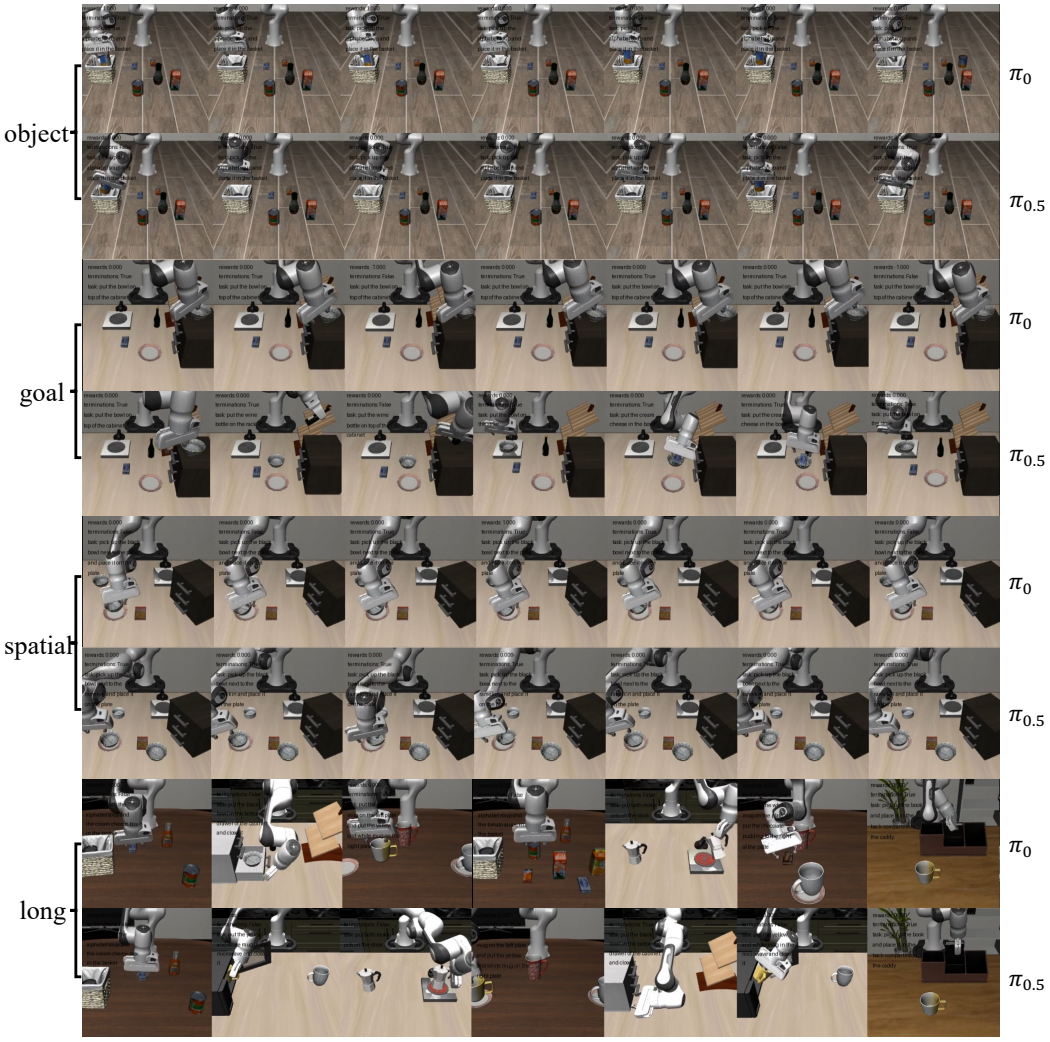


Figure 6: Additional simulation experiment results of PAWN on the LIBERO benchmark.

F Real-World Deployment Study

F.1 Overview

We conduct a real-world deployment study to evaluate whether PAWN can improve the OOD robustness of a flow-based VLA policy using fixed real robot data. We emphasize that this study does not perform real-world online RL: the robot is used only to collect a fixed batch of real trajectories, and all policy updates are performed offline. The goal is to test whether PAWN can better exploit real successes, failures, and human corrections than standard real-data fine-tuning.

The overall pipeline is

$$\theta_{\mathrm{FSFT}} \;\rightarrow\; \mathcal{D}_{\mathrm{real}} \;\rightarrow\; \theta_{\mathrm{real}} \;\rightarrow\; \text{held-out real OOD evaluation},$$

where θ_{FSFT} is a few-shot $\pi_{0.5}$-style initialization, $\mathcal{D}_{\mathrm{real}}$ is a fixed real robot batch, and θ_{real} is the policy obtained after offline real-world adaptation. This design isolates the effect of the adaptation objective: both the SFT baseline and PAWN start from the same initialization, use the same real data budget, and are evaluated under the same held-out real-world protocol.

F.2 Robot Platform and Tasks

We conduct experiments on a Piper COBOT MAGIC robot arm equipped with an RGB camera and a parallel gripper. The policy receives language-conditioned visual observations and proprioceptive states, and outputs Cartesian end-effector action chunks consisting of translation, rotation, and gripper actuation. This action representation matches the flow-based action interface used in our simulation experiments.

We evaluate two tabletop manipulation tasks:(A) *Pick up the small green wooden block and place it into the silver pot.* (B) *Pick up the round pot lid and place it on the silver pot.* Both require object localization, reaching, grasping, and goal-directed placement. Each episode starts from a randomized object pose within the robot workspace and terminates when the task is completed or when the maximum horizon is reached. A rollout is counted as successful if the target object is manipulated to the specified goal configuration without human intervention. All evaluation trials are manually reset.

F.3 Real Batch Collection

Starting from θ_{FSFT}, we collect a fixed real batch for offline adaptation. For each of the two tasks, we collect

$$20 \text{ successful teleoperated demos}, 30 \text{ autonomous rollouts}, 10 \text{ human intervention trajectories}.$$

The autonomous rollouts are executed without updating the model on the robot. They are then labeled as successes or failures according to the final task outcome. Human intervention trajectories are collected when the autonomous policy drifts toward failure; these trajectories provide local correction behavior around difficult states.

Table 9 summarizes the data budget. The resulting dataset is

$$\mathcal{D}_{\mathrm{real}} = \mathcal{D}_{\mathrm{demo}} \cup \mathcal{D}^{+}_{\mathrm{auto}} \cup \mathcal{D}^{-}_{\mathrm{auto}} \cup \mathcal{D}_{\mathrm{int}},$$

where $\mathcal{D}_{\mathrm{demo}}$ contains successful teleoperation data, $\mathcal{D}^{+}_{\mathrm{auto}}$ and $\mathcal{D}^{-}_{\mathrm{auto}}$ denote autonomous successes and failures, and $\mathcal{D}_{\mathrm{int}}$ contains intervention segments. For autonomous rollouts, we record the solver traces of generated action chunks when available; for teleoperated and intervention data, we reconstruct flow-matching training tuples by forward noising the recorded real action chunks.

F.4 Offline PAWN Adaptation

During real-world adaptation, we freeze the vision-language backbone and update only the action expert. This prevents the small real batch from overfitting the visual-language representation while

Data type	Per task	Two tasks	Role
Success demos	20	40	real action support
Autonomous rollouts	30	60	real success/failure feedback
Human interventions	10	20	local correction behavior
Held-out evaluation	20 per setting	40 per setting	clean/OOD testing

Table 9: Real-world data budget. The policy is fixed during real batch collection; all policy updates are performed offline.

allowing the action generator to adapt to real robot dynamics, camera geometry, and contact timing. We denote by c_i the full policy context, including visual observation, language instruction, and proprioception.

We use a conservative objective that combines real action imitation, negative-aware local ranking, pair guidance, intervention correction, and trust-region regularization:

$$\mathcal{L}_{\text{real}} = \mathcal{L}_{\text{FM}} + \lambda_{\text{rank}}\mathcal{L}_{\text{rank}}^{\text{sem}} + \lambda_{\text{pair}}\mathcal{L}_{\text{pair}}^{\text{real}} + \lambda_{\text{int}}\mathcal{L}_{\text{int}} + \lambda_{\text{TR}}\mathcal{L}_{\text{TR}}.$$

Flow-matching behavior cloning. For successful demonstrations and autonomous successes, we apply flow-matching behavior cloning over real action chunks:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{(c_i,a_i)\sim\mathcal{D}_{\text{demo}}\cup\mathcal{D}_{\text{auto}}^{+}}\left[\|v_\theta(x_{i,\rho},\rho,c_i) - u_{i,\rho}\|_2^2\right],$$

where a_i is the recorded real action chunk, $x_{i,\rho}$ is the corresponding noised solver state, and $u_{i,\rho}$ is the flow-matching target. This term anchors the adapted policy to physically valid real robot actions.

Real-batch negative-aware ranking. To use autonomous failures rather than discard them, we train a chunk-level scorer from real success/failure rollouts and construct a semantic negative-aware ranking loss. Preference pairs are formed within the same task and comparable reset-pose bins to reduce spurious ranking caused by task difficulty or initial-state mismatch. The ranking loss is

$$\mathcal{L}_{\text{rank}}^{\text{sem}} = \mathbb{E}_i\left[\text{softplus}\left(\frac{1}{2}y_{\phi,i}\Delta\mathcal{E}_i^{\text{sem}}\right)\right],$$

where $y_{\phi,i} \in [-1,1]$ is the chunk-level quality label predicted by the real-batch scorer, and $\Delta\mathcal{E}_i^{\text{sem}} := \bar{\delta}_i^{\text{sem}}$ is the motion-mode mirrored energy gap in Eq. (10). When $y_{\phi,i}$ is high, minimizing the loss encourages stronger support for the positive mirrored branch; when $y_{\phi,i}$ is low, it suppresses the corresponding local transition. This transfers PAWN's chunk-level local preference mechanism to real robot data.

Real-batch pair guidance. When the scorer can distinguish nearby candidate continuations, we further apply conditional pair guidance. From the same stored or reconstructed solver state, we internally sample multiple suffix chains, score the realized suffix paths with the learned real-batch scorer, and impose

$$\mathcal{L}_{\text{pair}}^{\text{real}} = w_{\text{pair},i}\,\text{softplus}\left(\frac{\kappa}{2}\left(\Delta\mathcal{E}_{i,j}^{g,\text{sem}} - \Delta\mathcal{E}_{i,j}^{b,\text{sem}}\right)\right).$$

This term is mainly applied to autonomous rollouts with available solver traces, or to reconstructed flow states from recorded action chunks. It does not require executing candidate continuations on the robot, preserving the low-interaction-cost property of the method.

Intervention correction. For human intervention data, we apply a correction loss only on intervention segments:

$$\mathcal{L}_{\text{int}} = \mathbb{E}_{(c_i,a_i^{\text{human}})\sim\mathcal{D}_{\text{int}}}\left[w_i\left\|v_\theta(x_{i,\rho},\rho,c_i) - u_{i,\rho}^{\text{human}}\right\|_2^2\right],$$

where w_i is larger around critical correction moments, such as wrong approach direction, unstable grasp timing, or near-drop recovery. This term teaches local recovery behavior without forcing the policy to imitate the entire human intervention distribution.

Method	T1 Clean	T1 OOD	T2 Clean	T2 OOD	Clean Total	OOD Total	Drop ↓
Real SFT adaptation	18/20	13/20	12/20	8/20	30/40	21/40	22.5
Offline π-StepNFT	17/20	15/20	14/20	10/20	31/40	25/40	15.0
Offline PAWN adaptation	18/20	16/20	14/20	12/20	32/40	28/40	10.0

Table 10: Held-out real-world evaluation on two tabletop tasks. All three methods start from the same few-shot $\pi_{0.5}$ policy, use the same fixed real-data budget, and are evaluated without online robot updates. The OOD setting introduces clutter, distractors, larger pose variation, and mild lighting changes while preserving the same task goals. Drop denotes the percentage-point decrease from clean to OOD success rate.

Trust-region regularization. To reduce overfitting to the small real batch, we regularize the action expert toward the initial few-shot policy:

$$\mathcal{L}_{\mathrm{TR}} = \mathbb{E}_{x,\rho,c}\left[\left\|v_\theta(x,\rho,c) - v_{\theta_{\mathrm{FSFT}}}(x,\rho,c)\right\|_2^2\right].$$

This velocity-level trust region preserves the general action prior of $\pi_{0.5}$ while allowing real-world correction from the collected batch.

For the SFT baseline, we use the same θ_{FSFT}, the same real-data budget, and the same number of offline update steps. The baseline performs standard real-data flow-matching adaptation on imitation and correction data, but does not use the negative-aware ranking or pair-guidance terms. Thus, differences in held-out performance reflect the adaptation objective rather than additional real robot interaction.

Matched offline π-StepNFT baseline. This baseline uses the same few-shot $\pi_{0.5}$ initialization, frozen VLM, action-expert update budget, fixed batch, and evaluation trials. It retains the flow-matching, intervention, and trust-region terms, and applies trajectory-labeled step-wise mirrored supervision on the identical stored or reconstructed solver transitions in place of PAWN's chunk, motion-mode, and pair terms.

F.5 Held-Out Real-World Evaluation

After offline adaptation, each policy is deployed without additional online updates. We evaluate under two settings. The *clean* setting matches the data-collection environment, with a plain tabletop, standard lighting, and moderate object-pose randomization. The *complex/OOD* setting preserves the same task semantics but introduces a textured background, distractor objects, larger target-pose randomization, and mild lighting variation. For each task and each setting, we run 20 held-out trials per method and report raw success counts. Figure 7 illustrates Task 1 execution.

F.6 Discussion

Table 10 reports both per-task success counts and aggregated clean/OOD performance. Compared with Real SFT adaptation, Offline PAWN adaptation improves the average OOD success rate from $21/40$ to $28/40$ and reduces the clean-to-OOD drop from 22.5 to 10.0 percentage points. Since both methods start from the same few-shot $\pi_{0.5}$ initialization and use the same real-data budget without online robot updates, the improvement suggests that PAWN provides a more effective offline adaptation objective for real-world OOD deployment.

We attribute this effect to three design choices. First, the chunk-level scorer converts sparse real success/failure labels into local quality signals. Second, motion-mode probing organizes updates into translation, rotation, and gripper subspaces, which better match real manipulation errors. Third, same-prefix pair guidance supplies a local preference direction without executing additional candidate actions on hardware. These properties allow PAWN to exploit real hard negatives and intervention corrections more effectively than imitation-only adaptation, while avoiding the safety and cost issues of real-world online RL.

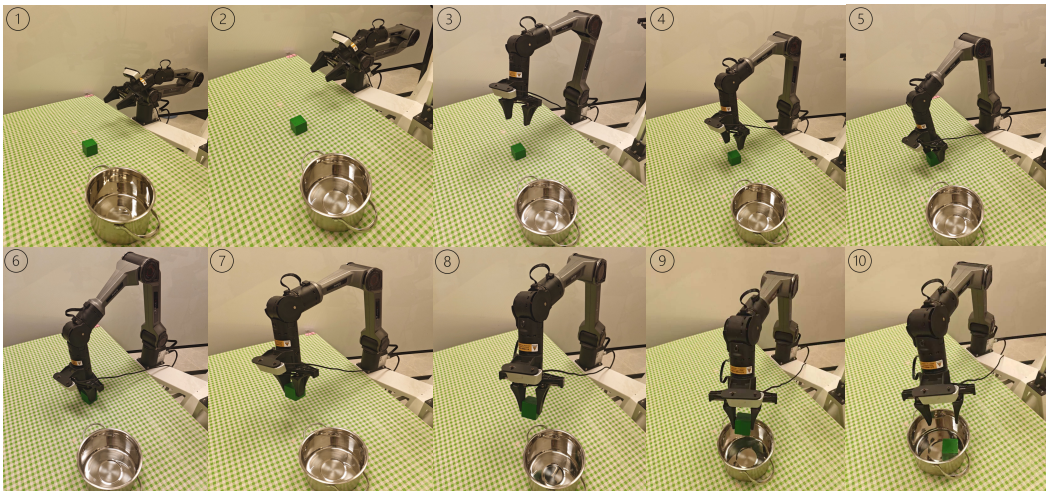
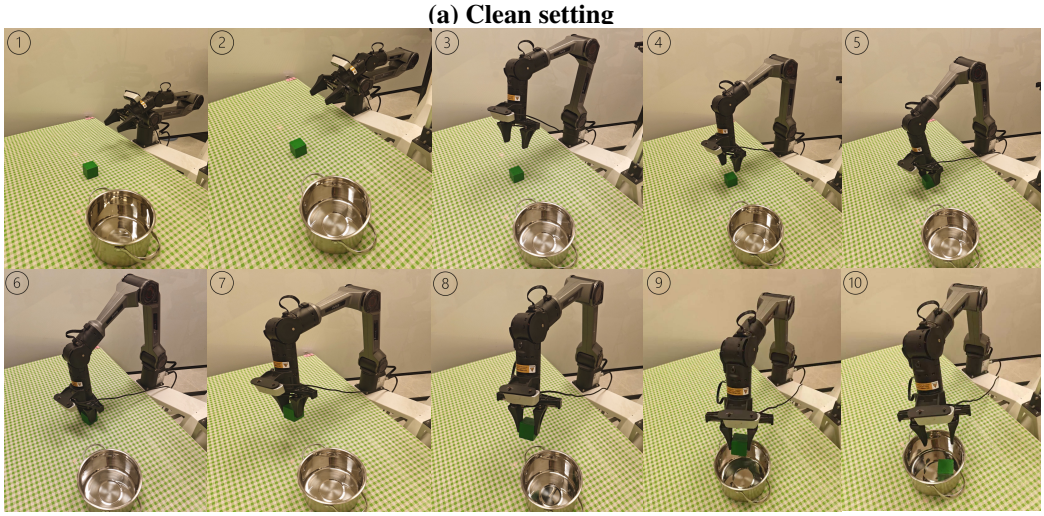


Figure 7: Real-world results of executing Task 1: *Pick up the small green wooden block and place it into the silver pot.*

The matched offline π-StepNFT control achieves $25/40$ OOD successes and a 15.0 pp clean-to-OOD drop; PAWN achieves $28/40$ and 10.0 pp. Clean success counts are $31/40$ and $32/40$, respectively. The study uses one fixed training batch and 20 held-out trials per task and setting.

(a) Clean setting



(b) Held-out OOD setting with clutter and distractors

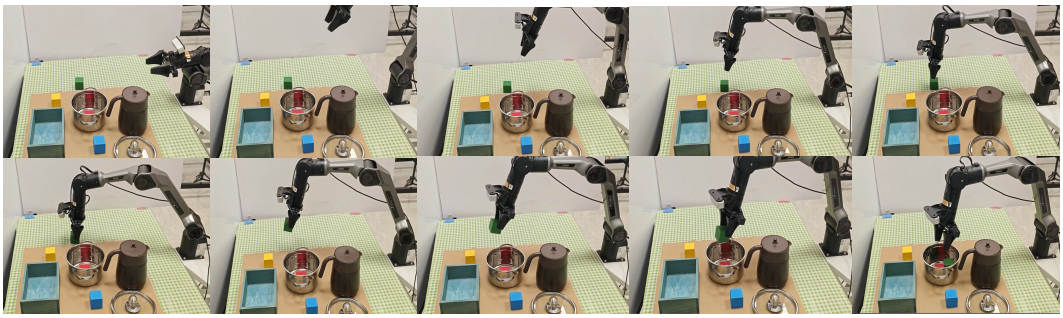
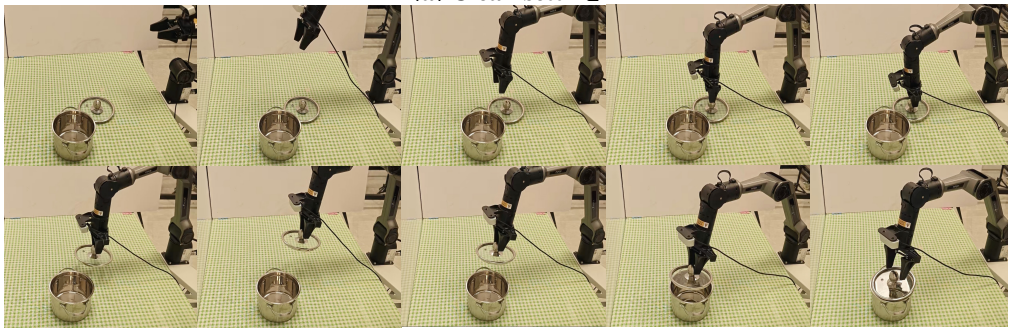


Figure 8: Real-world qualitative rollouts for Task 1: *Pick up the small green wooden block and place it into the silver pot*. The top sequence shows execution in the clean evaluation setting, while the bottom sequence shows a held-out OOD scene with clutter, distractor objects, and a changed scene layout. The adapted policy completes the full grasp-and-place behavior in both settings.

(a) Clean setting



(b) Held-out OOD setting with clutter and distractors

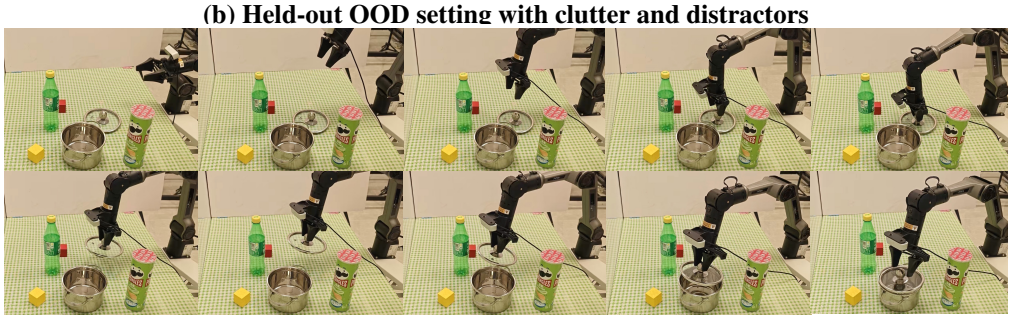


Figure 9: Real-world qualitative rollout for Task 2: *Pick up the round pot lid and place it on the silver pot*. The top sequence shows execution in a clean tabletop setting, while the bottom sequence shows a held-out OOD scene with visual clutter, distractor objects, and altered object layout. The adapted policy maintains the complete grasp-and-place behavior under the harder scene perturbation.